N056 원가를 계산하고 크레딧 결제 붙이기_해설서

Chapter01

챕터 01. 원가를 계산하고 크레딧 결제 붙이기

1. 강의의 목적

이번 강의의 핵심은 AI 에이전트를 기술적으로 작동시키는 단계에서 한 걸음 더 나아가, 실제 돈을 벌고 지속적으로 운영할 수 있는 서비스로 만드는 것입니다.

AI 친구 에이전트는 사용자가 메시지를 보낼 때마다 LLM API를 호출합니다. 호출할 때마다 비용이 발생하기 때문에 사용자가 많아질수록 매출이 없어도 운영비는 계속 증가합니다.

따라서 다음 네 가지를 결정해야 합니다.

1. 메시지 한 건을 처리하는 데 실제 얼마가 드는가?
2. 사용자에게 어느 정도까지 무료로 제공할 것인가?
3. 무료 한도를 초과하면 얼마를 받을 것인가?
4. 결제와 크레딧 지급을 어떻게 안전하게 처리할 것인가?

즉, AI 서비스의 원가 계산 → 가격 결정 → 크레딧 설계 → 결제 시스템 구축으로 이어지는 사업화 과정입니다.

2. 중학생도 이해하는 핵심 개념

① AI 서비스의 원가 계산

AI 에이전트는 사용자가 질문하면 AI 모델에 질문을 전달하고 답변을 받아옵니다.

이때 AI 모델을 사용하는 비용이 발생합니다.

예를 들어 다음과 같습니다.

- 입력 토큰: 사용자가 보낸 질문과 AI에게 전달하는 대화 기록
- 출력 토큰: AI가 생성한 답변
- 모델별 단가: GPT 등 모델마다 토큰당 사용료가 다름

비유: 음식점을 운영한다고 생각해 보겠습니다.

음식 한 그릇을 팔 때 재료비가 들어갑니다. AI 서비스도 메시지 하나를 처리할 때마다 AI 사용료라는 재료비가 들어가는 것입니다.

Langfuse는 이 사용 내역을 기록하고 분석하는 도구입니다.

② 사용자 유형별 월간 원가

모든 사용자가 똑같이 AI를 사용하는 것은 아닙니다.

사용자 유형	하루 메시지	월간 메시지
가벼운 사용자	10건	300건
일반 사용자	30건	900건
적극 사용자	100건	3,000건
과다 사용자	300건	9,000건

하루 300건을 사용하는 사람이 무료로 서비스를 이용한다면 사업자에게 상당한 비용 부담이 발생할 수 있습니다.

따라서 사용자별 사용량과 월간 원가를 계산해야 합니다.

③ 모델별 원가 비교

같은 질문을 두 가지 AI 모델에 전달하면 답변의 품질과 비용이 달라질 수 있습니다.

예를 들어 다음과 같은 비교가 가능합니다.

- 모델 A: 비용은 저렴하지만 답변 품질이 보통
- 모델 B: 비용은 비싸지만 답변 품질이 우수

중요한 것은 무조건 비싼 모델을 사용하는 것이 아닙니다.

서비스 품질을 유지하면서 원가를 낮출 수 있는 최적의 모델을 선택하는 것입니다.

이를 위해 Langfuse 기록을 활용해 실제 사용 비용과 답변 품질을 비교합니다.

④ 크레딧 가격 결정

AI 서비스를 운영하려면 원가뿐 아니라 결제 수수료와 고정비도 고려해야 합니다.

가격 결정에 고려할 요소는 다음과 같습니다.

항목	의미
AI 원가	메시지 처리에 필요한 AI 사용료
결제 수수료	PG사에 지불하는 결제 비용

항목	의미
고정비	서버, 데이터베이스 등 운영비
이익	지속적인 서비스 운영을 위한 수익

예를 들어 1,000원짜리 크레딧을 판매하더라도 실제로 사업자에게 1,000원이 남는 것은 아닙니다.

따라서 원가와 수수료를 계산한 후 적절한 크레딧 판매 가격을 결정해야 합니다.

⑤ 크레딧 원장 설계

여기서 중요한 기술적 개념이 등장합니다.

크레딧을 단순히 잔액 숫자 하나로 관리하면 문제가 발생할 수 있습니다.

예를 들어 잔액이 100이라고 기록되어 있다고 해도 언제 충전했고, 언제 사용했으며, 환불했는지 알 수 없습니다.

그래서 원장(Ledger) 방식으로 관리합니다.

거래 내용	크레딧 변화
최초 충전	+1,000
메시지 사용	-10
추가 충전	+500
메시지 사용	-10
현재 잔액	1,480

모든 충전과 사용 내역을 거래 기록으로 남기는 방식입니다.

또한 크레딧 차감은 반드시 서버에서 처리해야 합니다.

사용자 브라우저에서 직접 잔액을 수정할 수 있다면 부정 사용이 발생할 수 있기 때문입니다.

⑥ 토스페이먼츠 결제 연동

결제는 단순히 사용자가 결제 버튼을 누르면 크레딧을 지급하는 구조가 아닙니다.

안전한 결제 흐름이 필요합니다.

안전한 결제 처리 흐름

1. 사용자 결제 요청

   크레딧 상품을 선택하고 결제를 요청합니다.

2. 서버의 금액 검증

주문번호와 실제 결제 금액이 일치하는지 확인합니다.

3. 토스페이먼츠 승인

   서버가 결제 승인 요청을 전달합니다.

4. 결제 성공 확인

   결제 결과와 거래 정보를 검증합니다.

5. 크레딧 지급

   검증된 결제에 대해서만 크레딧을 원장에 기록합니다.

이번 강의에서는 토스페이먼츠 테스트 환경을 사용해 실제 결제와 동일한 흐름을 연습합니다.

3. 실생활 비유로 이해하기

이번 강의는 카페의 선불 충전 시스템을 만드는 것과 비슷합니다.

- AI API 원가 계산 = 커피 한 잔의 원재료비 계산
- 무료 한도 = 신규 고객에게 제공하는 무료 커피
- 크레딧 판매 = 선불 충전 카드 판매
- 크레딧 원장 = 고객별 충전 및 사용 내역
- 토스페이먼츠 = 카드 결제 시스템
- 서버 검증 = 돈이 실제로 들어왔는지 확인하는 직원

아무리 커피를 잘 만들어도 원가를 모르면 장사를 지속할 수 없습니다.

AI 서비스도 마찬가지입니다.

4. 실제 서비스에 적용한다면?

이번 내용은 현재 준비하고 있는 MSM(My-Story-Maker) 프로젝트에도 직접 적용할 수 있습니다.

예를 들어 MSM에서 사용자가 자신의 글을 웹소설로 변환한다고 가정하겠습니다.

MSM 기능	크레딧 차감 기준 예시
글 분석	분석 1회
문체 분석	분석 1회
웹소설 변환	변환 1회
문장 수정	수정 1회

MSM 기능	크레딧 차감 기준 예시
웹툰 시나리오 생성	생성 1회

각 기능은 사용하는 AI 모델과 토큰 수가 다르므로 기능별 원가를 측정해야 합니다.

이를 토대로 무료 체험, 월 구독, 크레딧 추가 충전 등의 수익 모델을 만들 수 있습니다.

특히 사용자가 글을 많이 입력하거나 긴 웹소설을 생성하는 경우에는 실제 사용량에 따라 크레딧을 차감하는 방식이 적합할 수 있습니다.

5. AI 서비스 개발자가 반드시 알아야 할 이유

이번 강의에서 중요한 것은 단순히 결제 API를 연결하는 기술이 아닙니다.

AI 서비스를 사업으로 운영하기 위한 기본적인 경제 구조를 이해하는 것입니다.

반드시 알아야 할 핵심은 다음과 같습니다.

1. 원가 측정: AI API 호출마다 얼마의 비용이 발생하는지 알아야 합니다.
2. 가격 정책: 원가보다 낮은 가격으로 서비스를 제공하면 사용자가 늘어날수록 손실도 커집니다.
3. 사용량 통제: 무료 한도와 유료 전환 기준을 정해야 합니다.
4. 거래 안정성: 결제가 중복되더라도 크레딧이 두 번 지급되지 않아야 합니다.
5. 보안: 클라이언트가 아니라 서버에서 금액과 크레딧을 검증하고 처리해야 합니다.

특히 중복 결제를 방지하는 멱등성(Idempotency)은 실제 서비스를 운영할 때 매우 중요한 개념입니다.

6. 학습 및 창업 목표와의 연결

이번 강의는 AI 기술 학습에서 AI 사업 운영 학습으로 넘어가는 중요한 단계입니다.

앞으로 만들려는 ASP의 AI 컨설팅 서비스, MSM, AI 콘텐츠 생성 서비스 등에도 공통으로 적용할 수 있습니다.

지금까지는 AI가 어떤 일을 할 수 있는지, 에이전트를 어떻게 만들고 배포하는지에 집중했다면 이제는 다음 질문을 해야 합니다.

- 고객 한 명을 확보하면 얼마의 비용이 발생하는가?
- 고객이 많아질수록 수익도 증가하는가?
- 무료 사용자와 유료 사용자를 어떻게 구분할 것인가?
- 고객의 사용량이 증가해도 손실이 발생하지 않는가?

- 결제와 사용량을 안전하게 관리할 수 있는가?

이는 1인 AI 창업자가 반드시 갖춰야 할 사업 운영 역량입니다.

7. 오늘 반드시 기억해야 할 핵심 정리

핵심 개념	반드시 기억할 내용
Langfuse	실제 LLM 사용량과 원가를 측정한다.
원가 분석	메시지 한 건의 비용과 사용자별 월간 비용을 계산한다.
모델 비교	비용과 답변 품질을 함께 비교한다.
가격 정책	원가, 결제 수수료, 고정비를 고려한다.
크레딧 원장	잔액 숫자뿐 아니라 모든 거래 내역을 기록한다.
서버 차감	크레딧 차감은 반드시 서버에서 처리한다.
결제 승인	서버에서 금액을 검증하고 결제 승인 후 크레딧을 지급한다.
멱등성	같은 결제 요청이 반복되어도 크레딧은 한 번만 지급한다.

8. 오늘의 강의에서 반드시 기억할 것

가장 중요한 세 가지

1. AI 서비스는 사용자가 증가할수록 API 원가도 증가하므로 정확한 원가 측정이 필요합니다.
2. 크레딧은 잔액 하나가 아니라 충전과 사용 이력을 기록하는 원장으로 관리해야 합니다.
3. 결제 성공 여부와 금액을 서버에서 검증한 후 크레딧을 지급해야 안전합니다.

9. 한 문장으로 정리

AI 에이전트를 만드는 것에서 끝나는 것이 아니라, 메시지 한 건의 원가를 측정하고 적절한 가격과 크레딧을 설계하며 안전한 결제 시스템까지 구축해야 비로소 지속 가능한 AI 비즈니스가 됩니다.

Chapter02

챕터 02. 시작 상태 맞추기와 메시지 한 건의 원가 계산

1. 강의의 목적

이번 강의의 핵심은 AI 에이전트가 메시지 하나를 처리하는 데 실제 얼마의 비용이 드는지 측정하고, 사용량에 따른 고객별 수익성과 손실 가능성을 계산하는 것입니다.

챕터 01에서 AI 서비스의 원가를 측정하고 크레딧 결제 시스템을 구축해야 하는 이유를 배웠다면, 이번 챕터에서는 실제 데이터를 이용해 원가를 계산합니다.

특히 중요한 관점이 있습니다.

> AI 서비스는 사용자가 많아서 비용이 증가하는 것이 아니라, 사용자가 AI를 많이 사용할수록 비용이 증가합니다.

따라서 평균적인 사용자의 원가만 계산해서는 안 됩니다. 많이 사용하는 사용자와 자동화 프로그램으로 과도하게 사용하는 계정까지 고려해야 합니다.

이번 강의는 크게 세 부분으로 구성됩니다.

1. 실습에 필요한 개발 환경과 서비스 상태 확인
2. Langfuse를 이용한 메시지 한 건의 원가 측정
3. 사용자 유형별 월간 원가와 수익성 분석

2. 실습 시작 상태 맞추기

이번 실습은 내 컴퓨터에서 실행하는 AI 친구 에이전트와 Supabase 테스트 프로젝트를 기반으로 합니다.

먼저 다음 환경이 준비되어 있어야 합니다.

준비 항목	확인할 내용	필요한 이유
로컬 저장소	Uvicorn으로 실행 가능	직접 코드를 실행하기 위해
Supabase	연결 및 익명 로그인	사용자와 크레딧 관리
LLM API	실제 모델 사용	실제 사용 비용 측정
Langfuse	트레이스와 비용 기록	호출별 원가 분석

실행 상태를 확인하는 명령어는 다음과 같습니다.

```
python -c "import httpx"
grep -c "^STORAGE=supabase" .env
grep -c "^LLM_PROVIDER=fake" .env
```

여기서 주의할 점은 `LLM_PROVIDER=fake`가 설정되어 있으면 실제 AI 모델을 호출하지 않는다는 것입니다.

가짜 모델은 개발과 테스트에는 유용하지만 실제 원가를 측정할 수는 없습니다.

핵심: 이번 실습은 실제 LLM API를 사용하고, 그 호출 내역을 Langfuse에서 확인하는 것이 기본입니다.

3. 메시지 한 건의 원가 이해하기

① 원가란 무엇인가?

원가(Cost)는 서비스를 고객에게 제공하기 위해 사업자가 부담하는 비용입니다.

AI 서비스의 원가는 크게 두 가지로 구분합니다.

구분	의미	AI 서비스 사례
변동비	사용량에 따라 증가	LLM API 호출 비용
고정비	사용량과 관계없이 발생하는 비용	서버, 데이터베이스, 관측 도구 등의 운영비

예를 들어 AI 친구 서비스에 사용자가 한 명 추가되더라도 서버 비용이 즉시 크게 늘지는 않을 수 있습니다. 하지만 메시지를 100개 보내면 LLM API 호출 비용은 그만큼 증가합니다.

따라서 이번 강의에서는 우선 변동비인 LLM API 원가를 측정하고, 서버 등의 고정비는 다음 가격 결정 단계에서 추가합니다.

② 메시지 하나에 LLM을 여러 번 호출한다

초보자가 놓치기 쉬운 중요한 부분입니다.

사용자가 메시지 하나를 보냈다고 해서 AI 모델이 반드시 한 번만 작동하는 것은 아닙니다.

AI 친구 에이전트는 다음과 같은 작업을 수행할 수 있습니다.

사용자 메시지 한 건의 처리 과정

사용자 메시지 입력

예: 오늘 기분이 별로야.

1. 도구 선택 호출

어떤 도구를 사용할지 판단합니다.

2. 답변 생성 호출

캐릭터 설정, 기억, 대화 기록 등을 참고해 답변합니다.

3. 기억 정리 호출

앞으로 기억할 만한 내용을 선택합니다.

최종 답변 및 기억 저장

각 호출마다 LLM API 비용이 발생합니다.

따라서 메시지 한 건의 총원가는 다음과 같습니다.

$$
C_{\text{message}} =
C_{\text{도구선택}}+
C_{\text{답변생성}}+
C_{\text{기억정리}}
$$

즉, LLM 호출 횟수와 각 호출의 비용을 모두 더해야 정확한 원가를 알 수 있습니다.

③ 토큰(Token)이 원가를 결정한다

토큰은 AI가 문장을 처리하는 기본적인 텍스트 단위입니다.

쉽게 말하면 AI가 읽는 글자 조각과 생성하는 글자 조각이라고 이해하시면 됩니다. 글자 수와 토큰 수가 정확하게 일치하지는 않습니다.

AI API 비용은 일반적으로 다음 두 가지로 결정됩니다.

- 입력 토큰(Input Tokens): AI에게 전달하는 질문과 참고 자료
- 출력 토큰(Output Tokens): AI가 생성하는 답변

계산식은 다음과 같습니다.

$$
\text{호출 비용} =
\frac{\text{입력 토큰}}{1,000,000}\times
\text{입력 단가}
+
\frac{\text{출력 토큰}}{1,000,000}\times
\text{출력 단가}
$$

예를 들어 입력 토큰 10,000개, 출력 토큰 2,000개를 사용했다고 가정하면 각각의 단가를 곱해 합산하면 됩니다.

여기서 중요한 사실이 있습니다.

사용자의 질문이 짧더라도 AI가 읽어야 하는 참고 자료가 많으면 비용이 증가합니다.

④ 대화가 길어지면 왜 비용이 증가할까?

예를 들어 사용자가 AI 친구에게 다음과 같이 대화한다고 생각해 보겠습니다.

- 첫 번째 메시지: 안녕?
- 두 번째 메시지: 오늘 날씨가 좋네.
- 세 번째 메시지: 어제 무슨 일이 있었는지 기억해?
- 열 번째 메시지: 아까 이야기한 내 고민을 다시 정리해 줘.

AI는 단순히 현재 질문만 읽는 것이 아닙니다.

캐릭터 설정, 기억, 과거 대화, 도구 결과 등을 함께 참고합니다.

따라서 대화가 길어질수록 입력 토큰이 늘어날 수 있습니다.

이번 실습의 템플릿은 과거 대화를 최근 10턴까지만 전달합니다. 이후에는 요약과 기억 메모가 중요한 역할을 합니다.

그러므로 다음 세 가지를 구분해서 측정해야 합니다.

1. 메시지 한 건의 평균 원가
2. 비용이 높은 상위 10% 메시지의 원가
3. 대화 첫 메시지와 열 번째 이후 메시지의 원가 차이

평균만 보면 실제로 비용이 많이 발생하는 사용자 유형을 놓칠 수 있기 때문입니다.

4. Langfuse로 실제 원가 측정하기

① Langfuse란?

Langfuse는 AI 서비스가 내부적으로 어떤 작업을 했는지 기록하고 분석하는 도구입니다.

일반적인 서버 로그가 사용자의 요청과 응답을 기록한다면, Langfuse는 AI 모델을 호출한 과정까지 관찰할 수 있습니다.

예를 들어 다음 내용을 확인할 수 있습니다.

항목	확인할 내용
Model	어떤 AI 모델을 사용했는가?
Input Tokens	AI가 읽은 토큰 수
Output Tokens	AI가 생성한 토큰 수
Cost	해당 호출에 발생한 비용
Trace	하나의 요청에서 발생한 전체 작업

항목	확인할 내용
Generation	실제 LLM 호출 단위

여기서 꼭 구분해야 할 개념이 있습니다.

- Trace: 사용자의 메시지 한 건을 처리하는 전체 과정
- Generation: 그 과정에서 실제로 발생한 개별 LLM 호출

예를 들어 메시지 한 건에서 LLM을 세 번 호출했다면 하나의 Trace 안에 세 개의 Generation이 기록될 수 있습니다.

따라서 메시지 원가를 계산할 때는 개별 Generation 비용을 모두 합산하거나, 전체 Trace의 비용을 확인해야 합니다.

② Langfuse에서 확인하는 방법

LLM Metrics & Analytics - Langfuse

langfuse.com

다음 순서로 확인합니다.

1. Langfuse에 접속합니다.
2. Tracing 메뉴에서 Traces를 선택합니다.
3. 조회 기간을 최근 7일로 설정합니다.
4. AI 친구 에이전트의 대화 기록만 필터링합니다.
5. 개별 Trace를 열어 Generation별 모델, 입력 토큰, 출력 토큰, 비용을 확인합니다.
6. 비용이 비어 있다면 프로젝트 설정의 Models에 해당 모델과 단가를 등록합니다.

주의할 점은 Langfuse가 모든 모델의 가격을 자동으로 알고 있는 것은 아니라는 사실입니다.

모델 이름이 가격표에 없으면 비용이 표시되지 않을 수 있습니다.

이때는 LLM 제공사의 공식 가격표를 확인해 모델 정의에 입력 단가와 출력 단가를 등록해야 합니다.

③ 실제 원가 기록표 만들기

이번 실습에서 반드시 작성해야 할 표입니다.

측정 항목	확인 목적
사용 모델	어떤 모델의 원가인지 구분

측정 항목	확인 목적
메시지 평균 비용	일반적인 메시지 원가
상위 10% 비용	비싼 메시지의 원가
가장 비싼 단계	비용을 많이 사용하는 작업 발견
첫 메시지 비용	대화 시작 시 원가
10번째 이후 비용	대화가 길어진 뒤의 원가

특히 모델 이름을 반드시 기록해야 합니다.

같은 서비스라도 저렴한 모델을 사용하는 경우와 고성능 모델을 사용하는 경우 원가가 크게 달라질 수 있기 때문입니다.

④ Langfuse 기록이 없다면?

아직 Langfuse에 비용 기록이 없다면 LLM 제공사의 Usage 화면을 이용할 수 있습니다.

실습 방법은 다음과 같습니다.

1. 실제 LLM API를 사용해 메시지를 정확히 20개 보냅니다.
2. 해당 시간대의 API 사용량을 확인합니다.
3. 입력 토큰과 출력 토큰, 총비용을 확인합니다.
4. 총비용을 20으로 나눕니다.

$$
\text{메시지 평균 원가} =
\frac{\text{20개 메시지 총비용}}{20}
$$

다만 이 방식은 전체 평균만 알 수 있습니다.

도구 선택, 답변 생성, 기억 정리 단계별 원가와 대화 길이에 따른 차이는 알기 어렵습니다.

5. 사용자 유형별 한 달 원가 계산

이번 챕터에서 사업적으로 가장 중요한 부분입니다.

AI 친구 서비스에는 사용량이 서로 다른 고객이 존재합니다.

예를 들어 하루에 20개 메시지를 보내는 고객과 300개를 보내는 고객은 사업자에게 발생시키는 비용이 전혀 다릅니다.

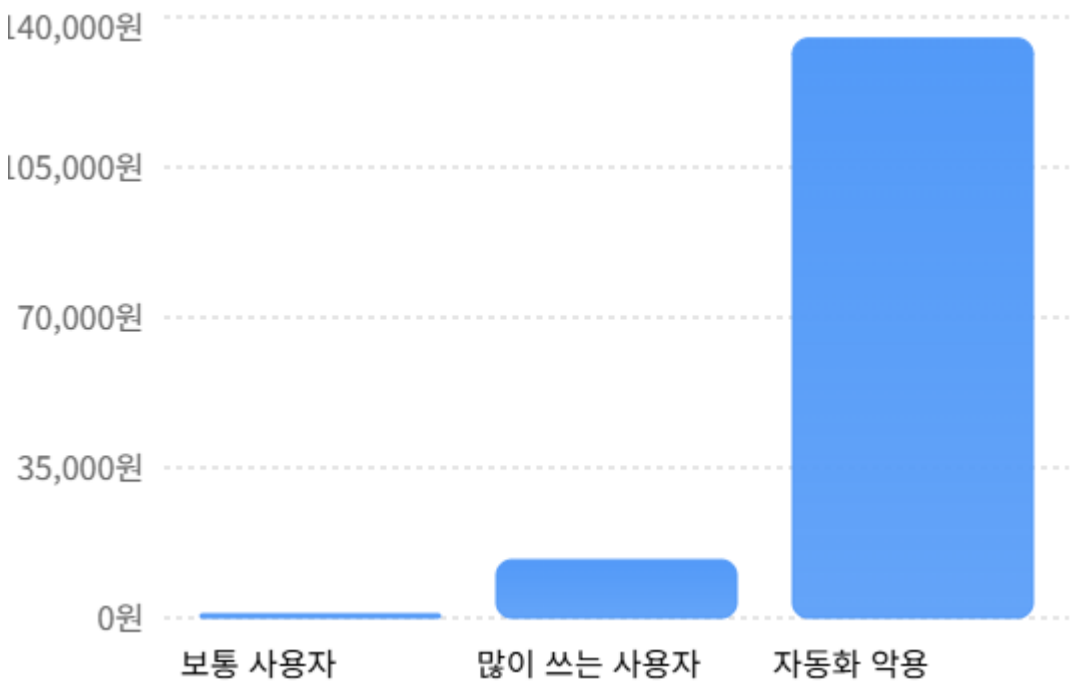
① 사용자 유형별 원가 계산식

$$
\text{월간 원가} =
\text{메시지당 원가}\times
\text{하루 메시지 수}\times30
$$

교안에서 제시한 예시인 메시지당 1.5원을 사용해 보겠습니다.

사용자 유형별 월간 원가

메시지당 원가 1.5원, 30일 기준의 예시



사용자 유형	하루 메시지	월간 메시지	월간 원가
보통 사용자	20건	600건	900원
많이 쓰는 사용자	300건	9,000건	13,500원
자동화 악용 계정	3,000건	90,000건	135,000원

여기서 중요한 점은 많이 사용하는 사용자와 악용 계정의 경우 평균 메시지 원가보다 열 번째 이후 메시지의 원가를 사용하는 편이 더 현실적이라는 것입니다.

② 왜 평균 원가만으로 가격을 정하면 안 될까?

보통 사용자의 월간 원가는 900원입니다.

사업자 입장에서는 상당히 저렴하게 느껴질 수 있습니다.

하지만 하루 3,000건을 보내는 계정은 월간 원가가 135,000원입니다.

한 명의 사용자 때문에 보통 사용자 150명분의 원가가 발생하는 것입니다.

이런 사용자가 여러 명이라면 서비스가 성장할수록 오히려 적자가 커질 수도 있습니다.

따라서 가격 정책은 평균 사용자만이 아니라 사용량 분포와 고사용자 비율까지 고려해서 결정해야 합니다.

6. 실생활 비유로 이해하기

AI 친구 서비스를 휴대전화 데이터 요금제에 비유하면 이해하기 쉽습니다.

- 메시지 한 건의 원가 = 데이터 1MB 사용료
- 무료 한도 = 매월 제공하는 무료 데이터
- 크레딧 충전 = 데이터 추가 구매
- 과다 사용자 = 데이터를 지나치게 많이 사용하는 고객
- 사용량 분석 = 고객별 데이터 사용량 확인

통신사가 모든 고객에게 무제한 데이터를 무료로 제공한다면 사업을 유지하기 어렵습니다.

AI 서비스도 마찬가지입니다.

고객에게 적절한 무료 사용량을 제공하면서 추가 사용량에 대해서는 요금을 받아야 합니다.

7. 내 프로젝트에 적용하기

이번 내용은 특히 MSM(My-Story-Maker)과 ASP의 AI 컨설팅 서비스에 직접 적용할 수 있습니다.

① MSM 적용 사례

MSM은 사용자의 글을 분석하고, 문체를 파악하고, 웹소설로 변환하고, 나중에는 웹툰까지 제작하는 서비스입니다.

각 기능별로 AI 원가가 다릅니다.

기능	예상 원가 발생 요인	가격 정책 방향
글 분석	입력 글의 길이	분석 횟수별 차감
문체 분석	글의 양과 분석 깊이	분석 크레딧
웹소설 변환	입력 글과 생성 분량	분량별 차감
문장 수정	수정 횟수와 대화 기록	수정 크레딧
웹툰 시나리오	생성 단계와 출력 길이	별도 크레딧

예를 들어 웹소설 1편을 생성하는 데 10번의 LLM 호출이 발생한다면, 10번의 호출 비용을 합산해야 합니다.

② ASP 컨설팅 서비스 적용

ASP에서도 AI 컨설팅 보고서를 생성할 때 원가를 측정할 수 있습니다.

- 간단한 사업 아이디어 분석
- 시장 조사 및 경쟁사 분석
- 사업계획서 작성
- 종합 전략 보고서 작성

이러한 작업은 AI 호출 횟수와 입력 자료의 양이 다릅니다.

따라서 단순 질문과 전문 컨설팅 보고서를 동일한 크레딧으로 판매하면 수익성이 나빠질 수 있습니다.

서비스 기능별 원가를 측정하고 기능별 크레딧을 차등 적용하는 것이 합리적입니다.

8. AI 개발자와 1인 창업자가 반드시 알아야 할 이유

이번 강의는 기술보다 사업적 관점에서 더 중요할 수 있습니다.

AI 서비스를 만드는 사람은 다음 세 가지 질문에 답할 수 있어야 합니다.

1. 고객 한 명이 한 달에 얼마의 비용을 발생시키는가?
2. 고객이 100명, 1,000명으로 증가하면 비용은 얼마나 증가하는가?
3. 많이 사용하는 고객이 늘어날 때도 수익을 유지할 수 있는가?

이를 위해서는 다음과 같은 운영 체계가 필요합니다.

영역	필요한 역량
관측	Langfuse로 AI 사용량 측정
분석	사용자별 비용과 사용량 분석
최적화	모델 변경 및 프롬프트 개선
사업	무료 한도와 유료 가격 설계
운영	과도한 사용량 제한 및 이상 계정 관리

특히 앞으로 AI 에이전트가 여러 개 연결되는 서비스에서는 한 번의 사용자 요청이 수십 번의 AI 호출을 발생시킬 수도 있습니다.

따라서 서비스 개발 초기부터 원가 측정 체계를 만들어야 합니다.

9. 오늘 반드시 기억해야 할 핵심 정리

핵심 개념 8가지

1. AI 서비스 원가는 변동비와 고정비로 나뉩니다.
2. 메시지 한 건에도 여러 번의 LLM 호출이 발생할 수 있습니다.
3. LLM 비용은 입력 토큰과 출력 토큰에 따라 달라집니다.
4. 대화가 길어지면 과거 대화와 기억 때문에 입력 토큰이 증가할 수 있습니다.
5. Langfuse의 Trace는 전체 요청이고 Generation은 개별 LLM 호출입니다.
6. 원가를 계산할 때 평균뿐 아니라 상위 10%의 비용도 확인해야 합니다.
7. 사용자 유형별 월간 원가를 계산해 과도한 사용에 대비해야 합니다.
8. 모델 이름과 환율, 측정 시점을 함께 기록해야 원가 데이터의 의미가 있습니다.

10. 오늘의 강의에서 반드시 기억할 것

이번 강의에서 가장 중요한 것은 AI 서비스의 원가는 고객 수가 아니라 실제 AI 사용량에 의해 결정된다는 사실입니다.

특히 다음 세 가지를 기억하시면 됩니다.

- 측정: Langfuse로 실제 메시지 처리 비용을 측정합니다.
- 분석: 평균 사용자뿐 아니라 고사용자와 악용 계정의 비용도 계산합니다.
- 대응: 사용량에 따라 무료 한도와 크레딧 가격을 다르게 설계합니다.

11. 한 문장으로 정리

지속 가능한 AI 서비스 사업을 만들려면 메시지 한 건의 원가부터 정확히 측정하고, 사용자별 사용량과 비용을 분석해 수익이 발생하는 가격 구조를 설계해야 합니다.

Chapter03

챕터 03. 모델에 따라 달라지는 원가와 크레딧 가격 결정

1. 강의의 목적

이번 강의의 핵심은 AI 모델의 성능과 비용을 비교하고, 그 결과를 바탕으로 서비스의 크레딧 차감 기준과 판매 가격, 무료 사용 한도를 결정하는 것입니다.

챕터 02에서는 현재 사용하는 AI 모델의 원가를 측정했습니다. 이번에는 한 단계 더 나아가 같은 메시지를 서로 다른 모델에 전달해 비용과 답변 품질을 비교합니다.

그리고 그 결과를 실제 사업의 가격 정책으로 연결합니다.

이번 강의의 학습 흐름은 다음과 같습니다.

1. AI 모델별 가격 차이 이해
2. 같은 메시지로 두 모델의 비용과 품질 비교
3. AI 서비스 원가를 줄이는 방법 학습
4. 크레딧 차감 단위 결정
5. 크레딧 상품 가격과 무료 한도 결정
6. 사용자 수와 사용량에 따른 손익 시뮬레이션

가장 중요한 관점은 다음과 같습니다.

> 좋은 AI 서비스를 만드는 것과 수익이 나는 AI 서비스를 만드는 것은 별개의 문제입니다. 성능과 비용 사이에서 적절한 균형을 찾아야 합니다.

2. AI 모델에 따라 원가가 달라지는 이유

① 같은 회사의 AI 모델도 가격이 다르다

AI 모델은 성능에 따라 가격이 다릅니다.

교안에서 제시한 2026년 11월 기준 OpenAI 가격을 살펴보겠습니다.

단위: 100만 토큰당 달러(교안 제시 가격)

모델	입력 토큰	출력 토큰
GPT-4o-mini	$0.15	$0.60
GPT-4.1	$2.00	$8.00
GPT-4o	$2.50	$10.00

GPT-4o는 GPT-4o-mini보다 입력과 출력 단가가 약 16.7배 비쌉니다.

즉, 같은 메시지를 처리하더라도 어떤 모델을 선택하느냐에 따라 서비스 원가가 크게 달라질 수 있습니다.

중학생도 이해하는 비유

자동차로 비유하면 다음과 같습니다.

- 작은 모델: 연비가 좋은 소형 자동차
- 큰 모델: 성능과 승차감이 좋은 고급 자동차

목적지까지 가는 데 반드시 비싼 자동차가 필요한 것은 아닙니다.

가까운 거리를 이동할 때는 소형 자동차로 충분하지만, 장거리 운행이나 높은 성능이 필요한 상황에서는 고급 자동차가 유리할 수 있습니다.

AI도 마찬가지입니다.

단순한 인사에는 저렴한 모델을 사용하고, 복잡한 추론이나 상담에는 성능이 높은 모델을 사용하는 것이 효율적입니다.

② 같은 메시지로 두 모델 비교하기

이번 실습의 핵심은 단순히 가격표를 비교하는 것이 아닙니다.

동일한 질문에 대해 두 모델이 어떤 답변을 하고, 각각 얼마의 비용이 발생하는지 직접 측정하는 것입니다.

실습 순서는 다음과 같습니다.

모델 비교 실습

1. 인사, 날씨, 고민 상담, 약속 등 메시지 10개를 준비합니다.
2. 현재 모델로 새 게스트 계정에서 10개를 순서대로 보냅니다.
3. `.env`의 `LLM_MODEL`을 큰 모델로 변경합니다.
4. 서버를 재시작하고 새 게스트 계정에서 똑같은 메시지 10개를 보냅니다.
5. Langfuse에서 두 사용자별 비용을 비교합니다.
6. 답변 품질까지 비교하고 원래 모델 설정으로 되돌립니다.

여기서 주의할 사항이 있습니다.

`.env` 파일의 모델 설정을 변경해도 서버가 자동으로 설정을 다시 읽지는 않습니다. 따라서 서버를 완전히 재시작해야 합니다.

비교 결과는 다음 표에 정리합니다.

비교 항목	현재 모델	큰 모델
모델 이름		
메시지 10개 비용		
메시지당 평균 비용		

비교 항목	현재 모델	큰 모델
답변이 더 좋았던 메시지 수		
품질 차이가 컸던 질문		

③ 결과를 어떻게 해석해야 할까?

일반적으로 모델 단가가 높아지면 원가도 증가합니다.

하지만 답변 길이가 달라질 수 있기 때문에 정확히 단가 배수만큼 비용이 증가하는 것은 아닙니다.

또한 모든 질문에서 큰 모델이 더 좋은 답을 주는 것도 아닙니다.

- 단순 인사: 작은 모델과 큰 모델의 차이가 작을 수 있습니다.
- 날씨 질문: 정보가 정확하게 주어졌다면 차이가 크지 않을 수 있습니다.
- 고민 상담: 문맥 이해와 공감 능력에서 차이가 나타날 수 있습니다.
- 복잡한 문제 해결: 추론 능력에 따라 차이가 커질 수 있습니다.

따라서 중요한 것은 큰 모델이 무조건 좋다는 결론이 아니라, 어떤 업무에서 큰 모델의 추가 비용을 지불할 가치가 있는지 판단하는 것입니다.

실제 서비스에 적용하기 전에는 10개 메시지의 눈으로 보는 비교뿐 아니라 평가 데이터셋을 이용해 품질을 검증해야 합니다.

3. AI 서비스 원가를 줄이는 다섯 가지 방법

모델을 무조건 저렴한 것으로 바꾸는 것만이 원가 절감 방법은 아닙니다.

다음 다섯 가지 방법을 이해해야 합니다.

방법	핵심 원리	주의할 점
단계별 모델 분리	작업에 따라 작은 모델과 큰 모델을 다르게 사용	작은 모델의 판단 오류
프롬프트 캐싱	반복되는 입력 내용을 캐시해 입력 비용 절감	제공사별 캐시 조건
출력 토큰 상한	AI 답변의 최대 길이 제한	답변이 중간에 끊길 수 있음
지난 대화 줄이기	과거 대화 입력량을 줄이고 요약 활용	과거 맥락 손실
기억 정리 빈도 조절	기억 정리 호출을 매번 하지 않고 간격을 둠	기억 반영 지연

① 단계별 모델 분리

제가 특히 중요하게 생각하는 방법입니다.

에이전트가 수행하는 모든 작업에 비싼 모델을 사용할 필요는 없습니다.

예를 들어 다음과 같이 구성할 수 있습니다.

에이전트 작업	권장 모델 방향
도구 선택	작은 모델
단순한 기억 정리	작은 모델
사용자에게 답변 생성	상황에 따라 큰 모델
복잡한 고민 상담	큰 모델

이는 MSM에도 적용할 수 있습니다.

글의 단순한 형식 검사에는 작은 모델을 사용하고, 사용자의 문체를 살리는 웹소설 변환에는 더 좋은 모델을 사용할 수 있습니다.

② 프롬프트 캐싱

매번 똑같이 전달하는 캐릭터 설정이나 시스템 지시문을 캐시해 두는 방법입니다.

예를 들어 AI 친구가 매번 다음 내용을 읽는다고 가정하겠습니다.

- 캐릭터의 성격
- 말투
- 행동 규칙
- 대화 원칙

이 내용은 매번 바뀌지 않습니다.

제공사가 지원하는 캐싱 기능을 사용하면 반복되는 입력에 대한 비용을 줄일 수 있습니다.

③ 출력 토큰 상한

AI가 너무 긴 답변을 생성하지 않도록 최대 출력 길이를 제한합니다.

출력 토큰이 많을수록 비용도 증가하기 때문입니다.

다만 친구처럼 대화하는 서비스에서 답변을 지나치게 짧게 제한하면 사용자 경험이 나빠질 수 있습니다.

④ 지난 대화 줄이기

대화가 길어지면 과거 대화가 계속 입력에 들어가면서 비용이 증가합니다.

최근 대화만 남기고 오래된 내용은 요약해서 전달하면 입력 토큰을 줄일 수 있습니다.

⑤ 기억 정리 빈도 조절

매번 메시지를 보낼 때마다 기억을 정리하는 대신 몇 번의 대화마다 한 번씩 정리하는 방법입니다.

기억 정리 호출 횟수가 줄어들면 비용도 줄어듭니다.

다만 중요한 정보가 늦게 반영될 수 있으므로 품질 검증이 필요합니다.

핵심: 비용 절감은 단순히 AI 모델을 저렴한 것으로 교체하는 것이 아니라, 서비스 품질을 유지하면서 불필요한 AI 호출과 토큰 사용을 줄이는 작업입니다.

4. 크레딧은 언제 차감해야 할까?

AI 서비스에 요금을 받기로 했다면 먼저 결정해야 할 것이 있습니다.

사용자가 어떤 행동을 할 때 크레딧을 몇 개 차감할 것인가?

이를 차감 단위라고 합니다.

교안에서는 세 가지 방식을 비교합니다.

차감 방식	사용자 입장	운영자 입장
메시지당 고정	비용을 예측하기 쉬움	비싼 메시지와 싼 메시지의 차이를 감수
사용 토큰만큼 차감	사용량만큼 정확하게 부담	원가와 잘 맞지만 사전 예측이 어려움
기본 고정 + 비싼 기능 추가	대부분 간단하고 예외만 추가 부담	원가가 높은 기능을 별도 관리

① 왜 메시지당 고정 차감을 선택했을까?

이번 AI 친구 서비스에서는 메시지 한 건당 크레딧 1개를 차감하기로 했습니다.

이유는 다음과 같습니다.

첫째, 사용자가 친구와 대화할 때마다 토큰 사용량을 신경 쓰게 만들면 자연스러운 대화가 어려워집니다.

둘째, 최근 대화 기록을 10턴까지만 전달하므로 대화가 길어져도 메시지당 원가가 무한정 증가하지 않습니다.

셋째, 사용자가 메시지 하나를 보낼 때 얼마의 크레딧이 차감되는지 쉽게 이해할 수 있습니다.

다만 예외가 있습니다.

- 일반 텍스트 대화: 1개
- 일정표 사진 분석: 추가 크레딧
- 고성능 AI 모델 선택: 추가 크레딧

즉, 기본 사용은 간단하게 만들고 원가가 크게 증가하는 기능에 대해서만 추가 차감하는 구조입니다.

5. 크레딧 가격을 결정하는 네 가지 재료

이제 원가를 측정하고 차감 단위도 결정했습니다.

다음은 실제로 얼마에 판매할 것인지 결정해야 합니다.

가격을 정할 때는 네 가지를 고려합니다.

① 메시지 한 건의 원가

가장 기본이 되는 비용입니다.

평균 원가만 사용하는 것보다는 많이 사용하는 고객에게서 발생하는 비용을 고려해야 합니다.

예를 들어 메시지 평균 원가가 1.5원이라도 대화가 길어지면 2원 이상 발생할 수 있습니다.

따라서 가격을 결정할 때는 평균보다 비싼 메시지의 원가를 참고하는 것이 안전합니다.

② 결제 수수료

교안에서는 2026년 11월 기준 토스페이먼츠 일반 등급 카드 결제 수수료를 다음과 같이 제시합니다.

- 기본 수수료: 3.4%
- 부가세: 기본 수수료의 10%
- 합산 부담률: 3.74%

예를 들어 1,000원을 결제받으면 약 37.4원이 수수료로 발생합니다.

결제 수수료도 상품 가격을 결정할 때 반드시 고려해야 합니다.

③ 고정비

사용자가 늘어나면 다음과 같은 서비스 운영비가 발생할 수 있습니다.

항목	교안 제시 비용
Supabase Pro	월 $25부터
Langfuse Core	월 $29부터
토스페이먼츠 가입비	220,000원
토스페이먼츠 연관리비	110,000원

실제 계약 조건과 요금제에 따라 달라질 수 있지만, 중요한 것은 고정비의 성격입니다.

사용자가 한 명이든 1,000명이든 일정한 비용이 발생합니다.

따라서 사용자가 적을 때는 한 명당 부담해야 하는 고정비가 커집니다.

④ 마진

마진은 원가와 수수료, 고정비 등을 제외하고 사업에 남기는 수익입니다.

AI 서비스를 운영하려면 단순히 메시지 원가만 회수해서는 안 됩니다.

무료 사용자에게 제공하는 비용과 서버 운영비까지 감당할 수 있어야 합니다.

6. 크레딧 상품 가격표 만들기

메시지 한 건의 원가가 1~2원 정도라고 가정해 보겠습니다.

이 경우 메시지 하나를 사용할 때마다 1원이나 2원을 결제하도록 만드는 것은 현실적으로 어렵습니다.

매번 결제하면 사용자의 불편도 커지고 결제 수수료도 발생하기 때문입니다.

그래서 크레딧을 묶음으로 판매합니다.

① 크레딧 상품 예시

상품 ID	판매 가격	지급 크레딧	크레딧 1개 가격
credits_100	1,000원	100개	10원
credits_550	5,000원	550개	약 9.1원
credits_1200	10,000원	1,200개	약 8.3원

이 구조는 많이 구매할수록 크레딧 하나의 가격이 저렴해지는 방식입니다.

예를 들어 하루 20개 메시지를 보내는 고객이라면 100개 크레딧은 약 5일 사용할 수 있습니다.

처음에는 두세 개의 상품으로 시작하는 것이 좋습니다.

② 반드시 확인해야 할 사항

여기서 중요한 것은 가장 큰 묶음의 크레딧 가격입니다.

큰 묶음일수록 크레딧 하나의 판매 가격이 낮아집니다.

그런데 고성능 모델을 사용하거나 대화가 길어져 메시지 원가가 높아진다면 크레딧 하나를 판매할 때 손실이 발생할 수도 있습니다.

따라서 다음을 확인해야 합니다.

- 가장 비싼 메시지의 원가

- 결제 수수료
- 크레딧 판매 가격
- 무료 사용량
- 고정비

그리고 결제 화면에서는 상품 ID만 서버로 전달해야 합니다.

예를 들어 사용자가 `credits_100`을 선택하면 서버가 상품표에서 1,000원과 100개라는 정보를 가져오는 방식입니다.

사용자 화면에서 가격과 크레딧 수를 직접 전달하도록 하면 악의적인 사용자가 금액을 조작할 수 있습니다.

핵심은 가격과 지급 크레딧의 결정권을 서버가 갖는 것입니다.

7. 무료 한도가 사업의 손익을 결정한다

이번 강의에서 사업적으로 가장 중요한 내용입니다.

AI 서비스는 유료 고객뿐 아니라 무료 고객도 원가를 발생시킵니다.

예를 들어 다음과 같은 상황을 가정해 보겠습니다.

- 전체 사용자: 1,000명
- 유료 전환율: 5%
- 유료 사용자: 50명
- 무료 사용자: 950명
- 메시지당 원가: 1.5원
- 무료 한도: 하루 10개

① 무료 사용자에게 발생하는 비용

무료 사용자 950명이 하루 10개씩 메시지를 보낸다면 다음과 같습니다.

$$
950 \times 10 \times 30 \times 1.5
$$

$$
\text{월간 무료 사용자 원가}=427,500\text{원}
$$

무료 사용자에게 제공하는 서비스만으로 매월 42만 7,500원의 비용이 발생합니다.

여기에 유료 사용자의 원가와 고정비까지 추가됩니다.

② 왜 크레딧 가격을 올리는 것만으로 해결되지 않을까?

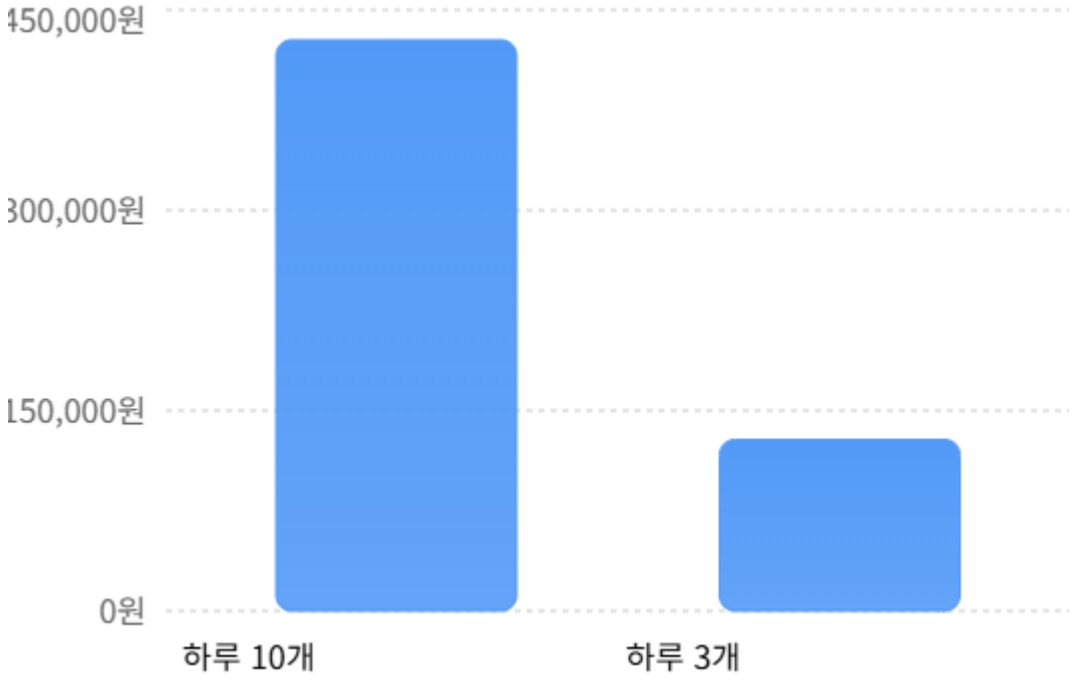
유료 고객이 50명이라면 크레딧 가격을 두 배로 올려도 영향을 받는 고객은 50명뿐입니다.

하지만 무료 고객 950명의 원가는 그대로 발생합니다.

반대로 무료 한도를 하루 10개에서 3개로 줄이면 무료 사용자에게 제공하는 원가 자체가 크게 줄어듭니다.

무료 한도 변경에 따른 월간 원가

무료 사용자 950명, 메시지당 1.5원, 매일 한도까지 사용하는 가정



무료 한도를 10개에서 3개로 줄이면 무료 사용자 원가가 427,500원에서 128,250원으로 감소합니다.

다만 무료 한도를 줄이면 신규 고객이 서비스를 충분히 경험하기 전에 떠날 수 있습니다.

따라서 무조건 무료 한도를 줄이는 것이 정답은 아닙니다.

무료 한도와 유료 전환율 사이의 균형을 찾아야 합니다.

③ 무료 사용자의 계정 중복 문제

또 하나 주의할 점이 있습니다.

무료 크레딧을 계정별로 지급하면 사용자가 게스트 계정을 반복해서 만들면서 무료 혜택을 계속 받을 수 있습니다.

따라서 실제 서비스를 운영할 때는 다음과 같은 통제 방법도 검토해야 합니다.

- 계정별 무료 크레딧 지급 제한
- 익명 계정의 반복 생성 방지
- 비정상적인 메시지 전송 패턴 감지
- 일정 시간 동안의 요청 횟수 제한

무료 정책은 단순히 몇 개를 줄 것인지뿐 아니라 누가 얼마나 반복해서 사용할 수 있는지도 함께 설계해야 합니다.

8. 원가와 가격을 직접 계산하는 이유

교안의 계산기는 다음 변수들을 바꾸면서 월간 손익을 확인하도록 설계되어 있습니다.

변수	의미
메시지 원가	메시지 하나를 처리하는 비용
전체 사용자	서비스 이용 고객 수
하루 메시지	고객별 사용량
무료 한도	무료로 제공하는 메시지 수
유료 전환율	무료 고객 중 결제하는 비율
크레딧 가격	크레딧 판매 가격
고정비	서버와 서비스 운영비

여기서 중요한 것은 크레딧 하나를 판매할 때 이익이 남더라도 전체 사업은 적자가 날 수 있다는 사실입니다.

예를 들어 다음과 같은 상황입니다.

- 유료 사용자 50명은 크레딧을 구매합니다.
- 무료 사용자 950명은 비용만 발생시킵니다.
- 유료 고객에게서 남긴 이익으로 무료 고객의 원가를 감당하지 못합니다.
- 서버와 데이터베이스 등의 고정비까지 추가됩니다.

결과적으로 개별 상품은 이익이 나지만 사업 전체는 적자가 됩니다.

이것이 단위경제성(Unit Economics)과 전체 손익을 함께 봐야 하는 이유입니다.

사업자가 기억해야 할 세 가지

1. 크레딧 가격을 올리면 유료 고객 한 명당 매출이 증가합니다.

2. 무료 한도를 줄이면 무료 고객에게 발생하는 원가가 감소합니다.
3. 유료 전환율이 높아지면 무료 고객의 원가를 감당할 수 있는 여력이 커집니다.

따라서 초기 서비스에서는 무료 한도를 보수적으로 설정하고 실제 사용량과 전환율을 측정하면서 조정하는 것이 좋습니다.

9. 내 프로젝트에 적용하기

이번 강의는 MSM과 ASP를 실제 유료 서비스로 전환할 때 매우 중요합니다.

① MSM(My-Story-Maker)

MSM은 일반적인 대화형 AI보다 기능별 원가 차이가 클 수 있습니다.

기능	모델 운영 전략	크레딧 정책
글 분석	작은 모델 우선	기본 차감
문체 분석	작은 모델 + 필요시 큰 모델	기본 차감
웹소설 변환	품질 중심 모델	분량별 추가 차감 검토
문장 수정	작은 모델 우선	기본 차감
웹툰 시나리오	큰 모델 활용	별도 크레딧

특히 MSM은 사용자가 자신의 글을 입력하는 서비스이므로 입력 토큰과 출력 토큰이 모두 많아질 수 있습니다.

따라서 메시지당 고정 차감만으로는 부족할 수 있습니다.

기능별 크레딧 정책이 더 적합합니다.

② ASP(AI Story Partner)

ASP는 고객의 사업계획서나 컨설팅 보고서를 작성하는 서비스입니다.

이때는 모든 AI 호출을 동일한 모델로 처리할 필요가 없습니다.

- 자료 분류: 작은 모델
- 자료 요약: 작은 모델
- 시장 분석: 상황에 따라 큰 모델
- 사업전략 수립: 큰 모델
- 최종 보고서 작성: 품질 중심 모델

이렇게 역할을 분리하면 컨설팅 품질을 유지하면서도 원가를 줄일 수 있습니다.

특히 고객에게 결과물 단위로 가격을 받는 경우에는 프로젝트 하나를 완료하는 데 발생하는 전체 AI 원가를 측정해야 합니다.

10. 오늘 반드시 기억해야 할 핵심 정리

핵심 개념	기억할 내용
모델별 가격	같은 제공사에서도 모델에 따라 원가가 크게 다르다.
모델 비교	같은 메시지로 비용과 답변 품질을 비교한다.
모델 분리	단순 작업은 작은 모델, 복잡한 작업은 큰 모델을 활용한다.
원가 절감	캐싱, 출력 제한, 대화 축약, 기억 정리 조절 등을 활용한다.
차감 단위	AI 친구 서비스는 메시지당 1크레딧으로 시작한다.
가격 결정	메시지 원가, 결제 수수료, 고정비, 마진을 고려한다.
상품 구성	크레딧을 여러 묶음으로 판매한다.
무료 정책	무료 한도와 유료 전환율이 사업의 손익에 큰 영향을 준다.
서버 보안	상품 ID만 전달하고 가격과 지급 크레딧은 서버에서 결정한다.

11. 객관식 퀴즈 정답과 해설

정답: 1번

무료 사용자 950명의 원가와 고정비를 유료 사용자 50명이 남기는 돈으로 메우지 못하기 때문입니다.

각 선택지를 살펴보겠습니다.

- 1번: 정답입니다. 무료 사용자가 많으면 무료 사용자의 원가가 누적되어 유료 고객의 이익을 초과할 수 있습니다.
- 2번: 오답입니다. 크레딧 1개는 10원이고 메시지 원가는 1.5원이므로 개별 판매에서는 원가를 초과합니다.
- 3번: 오답입니다. 결제 수수료 3.74%는 고려해야 하지만 전체 적자의 가장 큰 원인은 무료 사용자의 누적 원가입니다.
- 4번: 오답입니다. 사용자가 증가한다고 반드시 적자가 발생하는 것은 아닙니다. 가격과 사용량, 유료 전환율에 따라 수익이 날 수도 있습니다.

12. 오늘의 강의에서 반드시 기억할 것

이번 챕터의 핵심은 세 가지입니다.

첫째, AI 모델은 업무에 따라 다르게 사용해야 합니다.

모든 작업에 가장 비싼 모델을 사용할 필요는 없습니다. 성능이 필요한 곳에만 비용을 투자해야 합니다.

둘째, 크레딧 차감 기준과 상품 가격을 명확하게 설계해야 합니다.

사용자가 이해하기 쉽고, 사업자에게도 원가를 회수할 수 있는 구조여야 합니다.

셋째, 무료 한도가 사업의 수익성을 좌우합니다.

유료 고객이 늘어나는 것만큼 무료 고객의 사용량을 관리하는 것도 중요합니다.

13. 한 문장으로 정리

지속 가능한 AI 사업은 업무별로 적절한 AI 모델을 선택해 원가를 최적화하고, 크레딧 가격과 무료 사용 한도를 조정하여 고객 만족과 사업 수익성을 동시에 확보하는 것입니다.

Chapter04

챕터 04. 크레딧을 원장(Ledger)으로 설계하기

1. 강의의 목적

이번 강의의 핵심은 AI 서비스의 크레딧을 단순한 잔액 숫자가 아니라 모든 충전과 사용 내역을 기록하는 원장(Ledger) 방식으로 관리하는 것입니다.

챕터 01~03에서는 AI 서비스의 원가를 계산하고, 모델별 비용을 비교하고, 크레딧 가격과 무료 한도를 결정했습니다.

이번에는 실제로 크레딧을 관리하는 데이터베이스와 서버 구조를 구현합니다.

이번 강의의 학습 흐름은 다음과 같습니다.

1. 잔액 숫자 하나로 관리할 때의 문제 이해
2. 원장 방식의 개념과 장점 이해
3. Supabase에 크레딧 원장 테이블 만들기
4. 서버에서 크레딧 차감 및 잔액 관리
5. 크레딧 부족 시 HTTP 402 응답 처리
6. 테스트를 통해 정상 차감과 부족 상황 확인

가장 중요한 원칙은 다음과 같습니다.

> 크레딧의 잔액은 직접 수정하는 숫자가 아니라, 충전과 사용의 기록을 모두 더해서 계산하는 결과입니다.

2. 왜 잔액 숫자 하나로 관리하면 안 될까?

① 가장 쉬운 방법은 잔액 칸 하나를 만드는 것

사용자 테이블에 다음과 같은 잔액 항목을 추가한다고 가정하겠습니다.

사용자	크레딧 잔액
사용자 A	100
사용자 B	50
사용자 C	200

사용자가 메시지를 하나 보내면 잔액에서 1을 빼면 됩니다.

처음에는 아주 간단해 보입니다.

하지만 문제가 발생합니다.

- 크레딧이 언제 충전되었는지 알 수 없습니다.
- 언제 몇 개를 사용했는지 알 수 없습니다.
- 잘못 차감되었을 때 원인을 찾기 어렵습니다.
- 동시에 두 요청이 들어오면 잔액이 잘못 계산될 수 있습니다.
- 결제가 중복되어 크레딧이 두 번 지급되는 문제를 추적하기 어렵습니다.

특히 마지막 두 가지는 실제 서비스에서 매우 심각한 문제입니다.

② 동시에 요청하면 어떤 문제가 생길까?

예를 들어 사용자의 잔액이 1개 남았다고 가정하겠습니다.

사용자가 거의 동시에 메시지 두 개를 보냈습니다.

잔액 숫자 하나로 관리하는 경우

요청 A

잔액 1 확인

1 차감

잔액 0 저장

요청 B

잔액 1 확인

1 차감

잔액 0 저장

문제 발생

두 요청 모두 잔액이 있다고 판단해 메시지를 처리할 수 있습니다. 그런데 실제 잔액은 1개뿐입니다.

원장 방식

최초 잔액: 1

요청 A: -1 기록 성공

요청 B: 잔액 부족으로 거절

최종 잔액: 0

정상 처리

데이터베이스에서 원자적으로 잔액을 확인하고 차감하도록 설계하면 중복 사용을 막을 수 있습니다.

여기서 원자적(Atomic)이라는 개념이 중요합니다.

쉽게 말하면 잔액 확인과 차감을 하나의 작업처럼 처리한다는 뜻입니다.

중간에 다른 요청이 끼어들어 잘못된 결과가 발생하지 않도록 데이터베이스가 보호하는 것입니다.

3. 원장(Ledger)이란 무엇인가?

① 원장의 기본 개념

원장은 돈이나 포인트가 들어오고 나간 기록을 시간순으로 쌓는 장부입니다.

은행 계좌를 생각하면 이해하기 쉽습니다.

은행은 고객의 잔액만 기록하는 것이 아니라 입금, 출금, 이체 내역을 모두 기록합니다.

AI 서비스의 크레딧도 똑같이 관리합니다.

예를 들어 다음과 같습니다.

순서	거래 내용	크레딧 변화	잔액
1	신규 가입 무료 지급	+100	100
2	메시지 사용	-1	99
3	메시지 사용	-1	98

순서	거래 내용	크레딧 변화	잔액
4	크레딧 구매	+550	648
5	메시지 사용	-1	647
6	메시지 사용	-1	646

잔액은 원장에 기록된 모든 변화를 더한 값입니다.

$$
\text{현재 잔액}=\sum_{i=1}^{n}\text{거래별 크레딧 변화량}
$$

이 방식의 장점은 모든 거래의 근거가 남는다는 것입니다.

② 원장에 기록하는 다섯 가지 정보

이번 강의에서는 다음 정보를 기록합니다.

필드	의미	예시
user_id	누구의 거래인가?	user_001
변화량	크레딧이 얼마나 변했는가?	-1, +100
이유	왜 변했는가?	message, signup
참조	어떤 작업과 관련 있는가?	chat:123
만든 시각	언제 발생했는가?	2026-11-04

특히 중요한 것이 참조(Reference)입니다.

참조는 특정 거래를 식별하는 고유한 번호 또는 문자열입니다.

예를 들어 다음과 같습니다.

- 가입 무료 지급: `signup:user_001`
- 메시지 차감: `chat:message_123`
- 결제 충전: `payment:order_123`

참조를 유일하게 설정하면 동일한 작업이 반복되더라도 같은 거래가 중복 기록되지 않도록 데이터베이스가 막을 수 있습니다.

이를 고유 제약조건(Unique Constraint)이라고 합니다.

③ 원장은 수정하거나 삭제하지 않는다

원장 방식에서 매우 중요한 원칙입니다.

한 번 기록한 거래는 임의로 수정하거나 삭제하지 않습니다.

예를 들어 메시지 차감이 잘못되었다고 가정하겠습니다.

잘못된 기록:

거래	변화량
메시지 차감	-1

이때 -1을 삭제하는 것이 아니라 +1을 새로 기록합니다.

거래	변화량
메시지 차감	-1
차감 취소	+1
최종 변화량	0

이렇게 하면 잘못된 거래가 발생한 사실과 이를 바로잡은 기록이 모두 남습니다.

이것을 정정 거래 또는 보상 거래라고 이해하시면 됩니다.

4. 실제 구현 전에 결정해야 할 다섯 가지

이번 강의에서 중요한 점은 코딩 에이전트에게 모든 결정을 맡기지 않는다는 것입니다.

코딩 에이전트는 정해진 구조대로 코드를 작성하는 역할입니다. 사업 정책과 운영 기준은 서비스 운영자가 직접 결정해야 합니다.

① 언제 크레딧을 차감할 것인가?

두 가지 방식이 있습니다.

방식	장점	단점
메시지를 받자마자 차감	잔액 초과 사용 방지에 유리	답변 생성 실패 시 보상 처리 필요
답변 생성 후 차감	실패한 요청에 과금하지 않음	동시 요청에서 초과 사용 방지 필요

각각 장단점이 있습니다.

메시지를 받자마자 차감하면 사용자가 잔액보다 많은 요청을 동시에 보내는 문제를 막기 쉽습니다.

반면 답변을 만든 후 차감하면 실패한 요청에 대해서는 크레딧을 받지 않아도 됩니다.

② 답변 생성에 실패하면?

앞의 차감 시점과 연결되는 문제입니다.

- 차감하지 않는 방식: 답변 생성이 실패하면 애초에 차감하지 않습니다.

- 되돌리는 방식: 먼저 차감한 후 실패하면 +1을 기록합니다.

어떤 방식을 선택하든 실패 상황을 명확히 정의해야 합니다.

③ 크레딧이 0이면 어떻게 할까?

이번 강의에서는 HTTP 상태 코드 402를 사용합니다.

HTTP 402

Payment Required

크레딧이 부족해 결제가 필요한 상황을 나타냅니다.

```
{
  "error": "insufficient_credits",
  "message": "크레딧이 부족합니다. 충전 후 이용해 주세요.",
  "balance": 0
}
```

왜 402를 사용할까요?

상태 코드	의미	이번 상황에 적합한가?
200	정상 처리	아니요
401	인증 필요	아니요
402	결제 필요	적합
500	서버 내부 오류	아니요

화면에서는 402 응답을 받으면 크레딧 부족 안내와 충전 화면을 보여줄 수 있습니다.

④ 처음 온 사용자에게 무료 크레딧을 줄 것인가?

신규 사용자를 확보하기 위해 가입 시 무료 크레딧을 지급할 수 있습니다.

예를 들어 100개를 지급한다면 원장에는 다음과 같은 기록이 생깁니다.

사용자	변화량	이유	참조
user_001	+100	signup	signup:user_001

이렇게 하면 무료 크레딧 지급도 일반적인 충전과 마찬가지로 추적할 수 있습니다.

⑤ 어디에서 차감할 것인가?

핵심 원칙

서버에서만 차감합니다.

사용자의 브라우저 화면에서 잔액을 확인하고 차감하도록 만들면 안 됩니다.

왜냐하면 사용자가 개발자 도구를 이용해 화면 코드를 건너뛰고 서버 API를 직접 호출할 수 있기 때문입니다.

올바른 구조는 다음과 같습니다.

사용자 화면

잔액 표시 및 메시지 전송

서버 API (/api/chat)

로그인 확인 → 크레딧 확인 → 차감 → AI 호출

Supabase 원장

거래 기록 저장 및 잔액 관리

화면은 잔액을 보여 주기만 하고 실제 차감 권한은 서버가 갖습니다.

5. Supabase에 크레딧 원장 구현하기

이제 실제 데이터베이스에 원장을 만듭니다.

① 새로운 테이블 만들기

교안에서는 다음 마이그레이션 파일을 추가합니다.

```
supabase/migrations/0002_credits.sql
```

여기서 마이그레이션(Migration)이란 데이터베이스 구조를 변경하는 작업을 기록해 두는 파일입니다.

이번에는 `credit_ledger` 라는 테이블을 만듭니다.

이 테이블에는 다음 정보가 들어갑니다.

- user_id
- 변화량
- 이유
- 참조 문자열
- 만든 시각

또한 다음 두 가지를 적용합니다.

1. RLS(Row Level Security)를 활성화합니다.

2. 공개 정책을 만들지 않습니다.

RLS는 데이터베이스의 행마다 누가 읽고 수정할 수 있는지를 제한하는 보안 기능입니다.

이번 실습에서는 서버의 Secret Key를 통해서만 원장을 읽고 쓰도록 설계합니다.

즉, 사용자가 브라우저에서 직접 원장을 조작할 수 없도록 하는 것입니다.

② 저장소 계층에 동일한 기능 만들기

앱에는 메모리 저장소와 Supabase 저장소가 있습니다.

이번에는 두 저장소 모두 동일한 기능을 제공해야 합니다.

저장소	역할
메모리 저장소	로컬 개발 및 테스트
Supabase 저장소	실제 데이터 영구 저장

두 저장소에 같은 이름의 잔액 조회 및 기록 추가 함수를 구현합니다.

이렇게 하면 서비스의 나머지 코드는 저장소가 메모리인지 Supabase인지 크게 신경 쓰지 않아도 됩니다.

이것은 추상화(Abstraction)라는 소프트웨어 설계 개념과 연결됩니다.

쉽게 말하면 내부 구현 방식이 달라도 사용하는 방법은 동일하게 만드는 것입니다.

③ API 만들기

이번 강의에서는 두 가지 API가 중요합니다.

API	역할
POST /api/chat	메시지를 처리하면서 크레딧 차감
GET /api/credits	현재 잔액과 최근 기록 10개 반환

`GET /api/credits`는 잔액을 보여 주기 위한 API입니다.

화면은 이 API를 호출해 잔액을 확인합니다.

반면 실제 크레딧 차감은 `/api/chat`의 서버 로직에서 처리합니다.

④ 테스트하기

테스트에서는 가짜 LLM을 활용합니다.

가짜 LLM은 실제 API 비용을 발생시키지 않으면서도 메시지 처리 흐름을 확인할 수 있습니다.

테스트해야 할 내용은 다음과 같습니다.

1. 신규 사용자에게 무료 크레딧이 지급되는가?
2. 메시지를 하나 보내면 크레딧이 1개 줄어드는가?
3. 원장에 -1 기록이 남는가?
4. 잔액이 0이면 메시지가 거절되는가?
5. 기존 테스트도 정상적으로 통과하는가?

테스트 명령어는 다음과 같습니다.

```
pytest -q
```

마지막으로 Supabase SQL Editor와 Table Editor에서도 직접 확인합니다.

- `credit_ledger` 테이블이 만들어졌는가?
- RLS가 활성화되었는가?
- 신규 사용자에게 무료 크레딧이 지급되었는가?
- 메시지 전송 후 -1 기록이 남았는가?
- 잔액이 0이면 402 응답이 발생하는가?

6. 실생활 비유로 이해하기

원장은 은행 통장과 같습니다.

은행 통장에 100만 원이 남아 있다고 해서 은행이 잔액 숫자 하나만 관리하는 것은 아닙니다.

입금, 출금, 이체, 수수료 등 모든 거래가 기록되어 있습니다.

AI 크레딧도 동일합니다.

은행	AI 서비스
입금	크레딧 충전
출금	메시지 사용
잔액	남은 크레딧
거래 내역	크레딧 원장
거래 고유번호	참조 문자열
거래 취소	보상 거래

은행에서 거래 내역을 삭제하지 않는 것처럼 AI 서비스에서도 원장 기록을 계속 보존해야 합니다.

7. 오늘 배운 내용을 내 프로젝트에 적용한다면?

이번 내용은 앞으로 만들 MSM과 ASP의 결제 시스템을 설계할 때 공통 기반이 됩니다.

① MSM에 적용

MSM은 일반적인 메시지뿐 아니라 글 분석, 문체 분석, 웹소설 변환 등 다양한 기능을 제공합니다.

따라서 원장에 기능별 거래 이유를 구분해 기록하는 것이 좋습니다.

거래 이유	크레딧 변화 예시
신규 가입	+100
글 분석	-1
웹소설 변환	-10
문장 수정	-1
웹툰 시나리오 생성	-15
작업 실패 보상	+10

실제 차감 수량은 기능별 원가를 측정한 후 결정하면 됩니다.

이렇게 하면 어떤 기능에서 크레딧이 많이 소모되는지 분석할 수 있습니다.

② ASP에 적용

ASP 컨설팅 서비스는 프로젝트 단위로 AI 작업이 진행됩니다.

예를 들어 다음과 같은 거래가 발생할 수 있습니다.

- 사업 아이디어 분석
- 시장 조사
- 경쟁사 분석
- 사업계획서 작성
- 최종 보고서 생성

각 작업에 대한 원가와 크레딧 사용량을 원장에 남기면 프로젝트별 수익성을 계산할 수 있습니다.

특히 고객에게 컨설팅 비용을 받고 AI API 비용을 별도로 관리할 때 매우 유용합니다.

8. AI 개발자와 1인 창업자가 반드시 알아야 할 이유

이번 강의에서 배운 원장 설계는 단순한 크레딧 관리 기술이 아닙니다.

실제 돈을 다루는 모든 서비스의 기본 설계 원칙과 연결됩니다.

특히 다음 네 가지를 기억해야 합니다.

1. 거래 추적성: 모든 충전과 사용 내역을 기록해야 문제가 발생했을 때 원인을 찾을 수 있습니다.
2. 중복 방지: 고유 참조값을 사용해 같은 거래가 두 번 처리되지 않도록 해야 합니다.
3. 서버 중심 보안: 사용자가 직접 잔액을 조작하지 못하도록 서버에서 검증하고 차감해야 합니다.
4. 데이터 일관성: 동시에 여러 요청이 발생해도 잔액이 잘못 계산되지 않도록 데이터베이스의 원자적 처리가 필요합니다.

여기서 한 가지 더 중요한 개념이 있습니다.

원장에 고유 제약조건을 두는 것만으로 모든 동시성 문제가 해결되지는 않습니다.

실제 서비스에서는 잔액 확인과 차감 기록을 데이터베이스 트랜잭션이나 원자적인 RPC 함수 등으로 묶어 처리해야 합니다.

이 부분이 다음 단계의 결제 중복 방지와도 연결됩니다.

9. 오늘 반드시 기억해야 할 핵심 정리

핵심 개념	반드시 기억할 내용
잔액 관리	잔액 숫자 하나만 관리하면 거래 추적이 어렵다.
Ledger	크레딧의 증가와 감소를 거래 기록으로 쌓는다.
잔액 계산	모든 거래 변화량을 더해서 계산한다.
참조값	거래마다 고유한 참조값을 사용한다.
Unique	동일 거래의 중복 기록을 방지한다.
불변 원장	잘못된 기록은 삭제하지 않고 반대 거래로 정정한다.
RLS	데이터베이스에 대한 접근을 제한한다.
서버 차감	브라우저가 아니라 서버에서 크레딧을 차감한다.
HTTP 402	크레딧 부족 시 결제가 필요하다는 응답을 보낸다.
원자성	동시에 요청해도 크레딧이 초과 사용되지 않도록 한다.

10. 객관식 퀴즈 정답과 해설

정답: 2번

사용자가 화면 코드를 건너뛰고 서버 API를 직접 호출하면 크레딧 없이도 메시지를 보낼 수 있기 때문입니다.

각 선택지를 살펴보겠습니다.

- 1번: 버튼 디자인 문제이지 보안 문제는 아닙니다.
- 2번: 정답입니다. 브라우저는 사용자가 조작할 수 있으므로 서버에서 잔액을 확인하고 차감해야 합니다.
- 3번: 잔액 계산 속도와는 핵심적으로 관계가 없습니다.
- 4번: 화면에서 잔액을 관리한다고 원장을 사용할 수 없는 것은 아닙니다. 다만 서버에서 원장을 관리해야 안전합니다.

11. 더 해보기: 비싼 기능에 크레딧 더 받기

교안에서 제안한 확장 과제는 두 가지입니다.

① 일정표 사진 읽기 기능에 추가 크레딧 적용

사진을 읽는 OCR 기능은 일반 메시지보다 원가가 높을 수 있습니다.

따라서 Langfuse에서 사진 한 장을 분석하는 비용과 일반 메시지 비용을 비교하고, 적절한 크레딧을 차감하도록 구현합니다.

이때 일반 메시지와 OCR 거래의 참조값이 중복되지 않도록 서로 다른 접두사를 사용해야 합니다.

② 차감 개수를 한곳에서 관리하기

기능마다 코드에 숫자를 직접 입력하지 않고 설정 파일이나 서버 설정 한곳에 모아 관리합니다.

예를 들어 다음과 같은 구조입니다.

```
CREDIT_COSTS = {    "chat": 1,    "ocr": 5,    "webnovel": 10,    "webtoon_script": 15,}
```

이렇게 하면 원가가 변경되었을 때 여러 파일을 수정하지 않고 한곳에서 차감 정책을 변경할 수 있습니다.

12. 오늘의 강의에서 반드시 기억할 것

이번 강의의 핵심은 세 가지입니다.

- 기록: 크레딧을 단순 잔액이 아니라 거래 내역으로 관리합니다.
- 보안: 차감과 잔액 검증은 반드시 서버에서 처리합니다.
- 신뢰성: 고유 참조값과 원자적 처리를 통해 중복 차감과 초과 사용을 방지합니다.

13. 한 문장으로 정리

AI 서비스의 크레딧은 은행 통장처럼 모든 충전과 사용 내역을 원장으로 기록하고, 서버에서 안전하게 관리해야 신뢰할 수 있는 유료 서비스를 만들 수 있습니다.

Chapter05

챕터 05. 결제 대행사(PG)와 결제 승인

1. 오늘 강의의 목적

오늘 강의의 핵심은 사용자가 결제했다고 알려주는 것과 실제로 결제가 완료된 것은 다르다는 사실을 이해하는 것입니다.

AI 서비스를 운영하면서 크레딧을 판매하려면 사용자가 돈을 지불하고, 그에 맞는 크레딧을 정확하게 지급해야 합니다.

그런데 결제 과정에서 브라우저 화면만 믿으면 안 됩니다. 사용자가 화면의 금액을 바꾸거나 결제 성공 주소를 직접 호출할 수도 있기 때문입니다.

오늘은 토스페이먼츠를 예로 들어 다음 세 가지를 배웠습니다.

1. PG사가 무엇이고 왜 필요한가?
2. 결제 인증과 결제 승인은 어떻게 다른가?
3. 결제 승인 여부를 서버가 확인한 뒤에만 크레딧을 지급해야 하는 이유는 무엇인가?

2. PG(Payment Gateway)는 무엇인가요?

① PG를 사용하는 이유

온라인 쇼핑몰에서 카드 결제를 직접 처리하려면 카드사와 각각 계약하고, 카드 정보를 안전하게 처리하기 위한 복잡한 보안 요건도 충족해야 합니다.

이런 일을 대신해 주는 회사가 **PG(Payment Gateway, 결제 대행사)**입니다.

대표적인 예는 다음과 같습니다.

- 토스페이먼츠
- NHN KCP
- KG이니시스

Online payment process | 토스페이먼츠 개발자센터

docs.tosspayments.com

쉽게 비유하면 PG사는 온라인 상점과 카드사 사이에서 결제를 중개하는 금융 전문 대리인입니다.

참여자	역할
사용자	카드나 간편결제로 결제
우리 서비스	어떤 상품을 얼마에 판매할지 결정
PG사	결제창 제공, 카드 인증 및 승인 처리
카드사	실제 결제 가능 여부 확인 및 결제 처리

여기서 중요한 점은 우리 서비스가 카드 번호를 직접 다루지 않아도 된다는 것입니다.

② 토스페이먼츠 결제위젯

오늘 강의에서는 토스페이먼츠의 결제위젯을 사용했습니다.

결제위젯은 우리 서비스에 결제 화면을 제공하는 도구입니다.

사용자는 결제 수단을 선택하고 카드사 인증을 진행합니다. 우리 서비스는 카드 정보를 직접 보관하거나 처리하지 않고 결제 결과를 전달받습니다.

3. 결제 인증과 결제 승인은 다릅니다

오늘 강의에서 가장 중요한 개념입니다.

- *결제 인증(Authentication)**은 사용자가 카드사 인증을 완료했다는 의미이고, **결제 승인(Authorization/Confirm)**은 우리 서버가 PG사에 결제를 최종 확정해 달라고 요청하는 단계입니다.

결제의 두 단계

1단계 · 결제 인증

사용자가 결제창에서 카드 인증

카드사 인증을 마치고 결제 결과 정보를 전달받습니다.

아직 우리 서비스가 크레딧을 지급할 단계는 아닙니다.

2단계 · 결제 승인

우리 서버가 PG사에 최종 승인 요청

서버가 시크릿 키로 승인 API를 호출하고, PG사가 승인 성공을 응답하면 결제가 완료됩니다.

이때부터 크레딧 지급이 가능합니다.

중학생도 이해할 수 있도록 비유하면 다음과 같습니다.

- 결제 인증: 식당에서 손님이 카드와 신분을 확인받은 상태
- 결제 승인: 식당이 카드사에 실제 결제를 확정해 달라고 요청하고 승인받은 상태

즉, 카드 인증을 마쳤다고 해서 돈이 실제로 결제되었다고 판단하면 안 됩니다.

4. 실제 결제는 어떤 순서로 진행될까요?

오늘 강의의 1~9번 흐름을 이해하면 됩니다.

결제 흐름 이해하기 | 토스페이먼츠 개발자센터

docs.tosspayments.com

단계	처리 내용	핵심 의미
1	사용자가 크레딧 상품 선택	예: 5,000원 상품
2	서버가 주문을 DB에 저장	주문번호, 금액, 대기 상태 기록
3	주문 정보 전달	결제 준비
4	브라우저가 결제위젯 호출	카드 결제 화면 표시
5	인증 후 successUrl 이동	결제 키, 주문번호, 금액 전달
6	서버가 주문 정보 검증	저장된 주문 금액과 비교
7	서버가 PG사에 승인 요청	시크릿 키 사용
8	PG사가 승인 결과 반환	결제 성공 여부 확인
9	주문 완료 및 크레딧 지급	원장에 크레딧 거래 기록

가장 중요한 6번: 서버가 금액을 확인해야 합니다

예를 들어 사용자가 5,000원짜리 상품을 구매한다고 가정하겠습니다.

우리 서버는 결제를 시작하기 전에 다음 정보를 DB에 저장합니다.

```
주문번호: ORDER-001
상품: 크레딧 550개
결제금액: 5,000원
주문상태: 대기
```

결제 인증이 끝나면 successUrl에 다음과 같은 정보가 전달됩니다.

```
paymentKey=결제키
orderId=ORDER-001
amount=5000
```

그런데 사용자가 주소창의 금액을 5,000원에서 100원으로 바꿀 수도 있습니다.

```
amount=100
```

따라서 서버는 브라우저가 전달한 금액을 그대로 믿으면 안 됩니다.

서버에 저장된 주문 금액 5,000원과 전달받은 금액이 같은지 비교해야 합니다.

- 금액이 같고 주문 상태가 대기라면 승인 진행
- 금액이 다르면 승인 요청 중단
- 이미 완료된 주문이라면 중복 처리 방지

결제 금액의 기준은 브라우저가 아니라 우리 서버의 데이터베이스에 있어야 합니다.

5. 결제 보안에서 반드시 지켜야 할 5가지

원칙	이유
주문을 먼저 서버에 저장	결제 금액을 검증할 기준 확보
successUrl의 금액 검증	브라우저에서 금액을 조작할 수 있음
승인 요청은 서버에서 실행	시크릿 키 보호
승인 성공 후 크레딧 지급	인증만으로는 결제 완료가 아님
같은 주문은 한 번만 처리	새로고침이나 재시도로 중복 지급 방지

클라이언트 키와 시크릿 키의 차이

토스페이먼츠는 두 종류의 키를 사용합니다.

구분	클라이언트 키	시크릿 키
사용 위치	브라우저 화면	서버
사용 목적	결제위젯 실행	결제 승인 API 호출
공개 여부	공개 가능	절대 공개 금지
보관 위치	화면 코드	`.env`, Secret Manager

비유하면 클라이언트 키는 결제 창을 열 수 있는 출입증이고, 시크릿 키는 실제 돈을 움직일 수 있는 관리자 열쇠입니다.

시크릿 키가 화면 코드나 GitHub 저장소에 노출되면 다른 사람이 결제 관련 API를 악용할 위험이 있습니다.

중복 결제와 멱등성(Idempotency)

사용자가 결제 버튼을 두 번 누르거나 네트워크 오류 때문에 서버가 같은 승인 요청을 반복할 수 있습니다.

이때 같은 주문을 두 번 처리하면 크레딧이 중복 지급될 수 있습니다.

이를 막기 위해 다음과 같은 장치가 필요합니다.

- 주문번호와 결제 정보를 이용한 중복 검증
- 주문 상태 확인
- DB의 고유 제약조건
- PG사의 멱등키(Idempotency-Key) 활용

특히 DB에서 주문 완료 처리와 크레딧 지급을 하나의 트랜잭션으로 관리해야 합니다. 그래야 결제는 완료됐는데 크레딧이 지급되지 않거나, 반대로 중복 지급되는 문제를 줄일 수 있습니다.

6. PG사는 어떤 기준으로 선택할까요?

모든 결제 대행사가 같은 역할을 하지만 사업 형태에 따라 선택 기준이 달라집니다.

사업 상황	선택 가능한 서비스	특징
국내 일반 온라인 결제	토스페이먼츠	결제위젯, API 연동
여러 PG사를 유연하게 사용	포트원	여러 PG 연동을 통합 관리
해외 판매 및 세금 처리	Paddle	MoR 방식으로 판매·세금 업무 대행
모바일 앱 내 디지털 상품 판매	Google Play 결제 / Apple 인앱 결제	앱스토어 결제 정책 고려

MoR(Merchant of Record)은 쉽게 말해 결제 대행사가 단순한 결제 중개를 넘어 판매 주체 역할까지 맡는 방식입니다.

다만 각 서비스의 수수료와 계약 조건, 국가별 정책은 다르므로 실제 사업을 시작할 때 확인해야 합니다.

어떤 PG사를 선택하더라도 결제의 기본 원칙은 같습니다.

주문 저장 → 결제 인증 → 서버 검증 → 결제 승인 → 크레딧 지급

7. 지금까지 배운 크레딧 시스템과 연결하기

이번 강의는 챕터 04의 크레딧 원장 설계와 직접 연결됩니다.

지금까지의 흐름을 하나로 묶으면 다음과 같습니다.

AI 서비스의 크레딧 결제 구조

1. 상품 선택

   사용자가 5,000원짜리 크레딧 550개 상품 선택

2. 주문 생성

   서버 DB에 주문번호와 금액을 저장

3. PG 결제 인증

   토스페이먼츠 결제위젯에서 카드 인증

4. 서버 검증 및 승인

   주문 금액과 상태 확인 후 PG사에 승인 요청

5. 크레딧 원장 기록

   결제 승인 확인 후 +550 크레딧 기록

6. AI 서비스 이용

   사용자가 AI 서비스를 이용할 때마다 원장에 -1 또는 정해진 크레딧 기록

MSM(My-Story-Maker)에 적용한다면?

MSM은 사용자가 자신의 글을 입력하고 AI를 통해 분석하거나 웹소설로 변환하는 서비스입니다.

기능마다 AI 사용 비용이 다르므로 크레딧 상품과 결제 시스템을 연결할 필요가 있습니다.

MSM 기능	크레딧 사용 예시
글 분석	1
문장 다듬기	1
글을 웹소설로 변환	5
긴 원고 전체 변환	10
웹툰용 장면·대사 생성	5

위 수치는 시스템 설계를 설명하기 위한 예시이며, 실제 원가 측정 후 결정해야 합니다.

예를 들어 사용자가 10,000원 상품을 구매하면 다음 순서로 처리합니다.

1. 서버가 10,000원 주문을 생성합니다.
2. 토스페이먼츠에서 카드 인증을 진행합니다.
3. 서버가 주문 금액을 확인합니다.
4. PG사 승인 성공을 확인합니다.
5. 크레딧 원장에 +1,200을 기록합니다.
6. 사용자가 MSM에서 글을 변환하면 원장에 사용량을 차감합니다.

이렇게 해야 결제와 AI 사용량이 정확하게 연결됩니다.

ASP(AI Story Partner)에 적용한다면?

ASP 컨설팅 서비스에서도 동일한 구조를 사용할 수 있습니다.

- 기본 사업 아이디어 진단: 10크레딧
- 경쟁사 분석: 20크레딧
- 시장 분석 보고서: 30크레딧
- 종합 사업계획서 생성: 100크레딧

이처럼 AI 작업별 원가를 측정하고 크레딧을 책정하면 서비스의 수익성을 관리할 수 있습니다.

8. 객관식 퀴즈 정답과 해설

정답: 1번

문제: 결제 성공 페이지가 열리면 화면 코드가 바로 `/api/credits/add`를 호출해 크레딧 100개를 늘리도록 만들었습니다. 무엇이 문제일까요?

정답은 1번입니다.

이유는 두 가지입니다.

1. successUrl이 열렸다는 것은 카드 인증이 끝났다는 뜻이지, 결제 승인이 완료됐다는 뜻은 아닙니다.
2. 화면 코드에서 크레딧 지급 API를 직접 호출할 수 있다면 사용자가 결제하지 않고도 API를 호출해 크레딧을 늘릴 수 있습니다.

따라서 크레딧 지급은 반드시 서버가 PG사의 결제 승인 성공을 확인한 다음 실행해야 합니다.

9. 오늘 반드시 기억해야 할 핵심 내용

핵심 개념	기억할 내용
PG	카드사와 우리 서비스를 연결하는 결제 대행사
결제 인증	카드사 인증을 완료한 단계
결제 승인	서버가 PG사에 결제를 최종 확정하는 단계
successUrl	결제 인증 후 브라우저가 이동하는 주소
서버 검증	주문 금액과 상태를 DB 기준으로 확인
클라이언트 키	브라우저에서 사용 가능
시크릿 키	서버에서만 사용
멱등성	같은 결제 요청을 여러 번 처리하지 않도록 방지
크레딧 지급	PG 승인 성공 후에만 실행

10. 한 문장으로 정리

결제 시스템에서 가장 중요한 원칙은 사용자가 결제했다고 알려주는 화면을 믿는 것이 아니라, 우리 서버가 PG사의 승인 결과를 확인한 뒤에만 크레딧을 지급하는 것입니다.

11. 이번 강의가 창업 준비에 주는 의미

이번 강의는 단순히 카드 결제 기능을 만드는 기술을 배운 것이 아닙니다.

AI 서비스를 실제 사업으로 운영하기 위해 필요한 매출 발생 → 결제 확인 → 크레딧 지급 → AI 사용량 차감 → 원가 관리의 전체 구조를 배운 것입니다.

앞으로 MSM이나 ASP를 실제 유료 서비스로 만들 때 결제 기능은 단순한 부가 기능이 아니라 사업의 신뢰성과 수익성을 지키는 핵심 기반이 됩니다.

특히 기억하실 점은 다음과 같습니다.

- 결제는 PG사에 맡기되, 주문과 크레딧은 우리 서버가 관리합니다.
- 돈이 실제로 승인되었는지 서버가 확인해야 합니다.
- 크레딧 지급과 차감은 원장을 통해 기록해야 합니다.
- 중복 결제와 중복 지급을 방지해야 합니다.

지금까지 배운 크레딧 원장과 오늘 배운 PG 결제 승인을 연결하면, MSM과 ASP에서 실제 돈을 받고 AI 서비스를 제공할 수 있는 기본적인 상용화 구조가 만들어집니다.

Chapter06

챕터 06. 테스트 키 준비와 결제 시스템 구현

1. 오늘 강의의 목적

오늘 강의의 핵심은 실제 돈을 사용하지 않고 테스트 결제 환경을 구축하고, AI 코딩 에이전트를 활용해 안전한 결제 시스템을 구현하는 방법을 배우는 것입니다.

지난 챕터 05에서는 PG사와 결제 인증·승인의 차이를 배웠습니다. 이번 챕터에서는 그 내용을 실제 코드로 구현하기 위한 준비와 점검 방법을 배웁니다.

오늘 학습은 크게 네 가지입니다.

1. 토스페이먼츠 테스트 키를 준비하고 안전하게 관리하기
2. MCP를 이용해 AI 코딩 에이전트가 공식 결제 문서를 참고하도록 만들기
3. 크레딧 구매부터 결제 승인, 크레딧 지급까지 시스템 구현하기
4. 결제 결과를 세 곳에서 확인하고 배포 환경에 적용하기

2. 테스트 키란 무엇인가요?

① 실제 결제 키와 테스트 키의 차이

온라인 서비스에서 실제 카드 결제를 하려면 PG사와 전자결제 계약을 맺고 승인받아야 합니다.

하지만 개발 단계에서 매번 실제 돈을 결제하면서 테스트할 수는 없습니다.

그래서 토스페이먼츠는 테스트 키를 제공합니다.

토스페이먼츠 키의 종류

클라이언트 키

`test_gck`로 시작

결제위젯을 화면에 표시할 때 사용합니다. 브라우저에 전달해도 되는 키입니다.

시크릿 키

`test_gsk`로 시작

서버가 결제 승인이나 취소 API를 호출할 때 사용합니다. 외부에 노출하면 안 됩니다.

테스트 키로 결제하면 결제 승인은 가상으로 처리되므로 실제 돈이 빠져나가지 않습니다.

② 문서용 테스트 키의 특징

오늘 강의에서는 내 상점의 고유 키가 아니라 토스페이먼츠 공식 문서에 공개된 문서용 테스트 키를 사용합니다.

일반적인 테스트 키와 차이가 두 가지 있습니다.

구분	일반 테스트 키	문서용 테스트 키
발급	내 테스트 상점	공식 문서에 공개
결제 결과 확인	개발자센터에서 확인 가능	개발자센터 결제 내역에 남지 않음
시크릿 키	비공개	문서에 공개
실제 결제	테스트 결제	테스트 결제

문서용 테스트 키로 결제하면 개발자센터에서 거래 내역을 확인할 수 없습니다.

따라서 우리 서버가 받은 승인 응답과 `orders` 테이블을 통해 결제 결과를 확인해야 합니다.

중요한 점은 문서용 시크릿 키도 실제 시크릿 키와 똑같이 관리하는 습관을 들이는 것입니다.

나중에 내 상점의 키로 바꿔도 코드 구조는 그대로 유지하고 환경변수의 값만 변경할 수 있도록 만드는 것이 좋습니다.

3. 환경변수(.env)로 키를 관리하는 이유

① 환경변수란?

환경변수는 프로그램을 실행할 때 필요한 설정값을 코드와 분리해서 보관하는 방법입니다.

예를 들어 결제 키를 소스코드에 직접 입력하면 다음과 같은 문제가 발생합니다.

```
# 잘못된 방식TOSS_SECRET_KEY = "test_gsk_실제키값"
```

이 코드가 GitHub에 올라가면 다른 사람이 시크릿 키를 볼 수 있습니다.

따라서 다음과 같이 관리합니다.

`.env` 파일:

```
TOSS_CLIENT_KEY=test_gck_...
```

```
TOSS_SECRET_KEY=test_gsk_...
```

`.env.example` 파일:

```
TOSS_CLIENT_KEY=
TOSS_SECRET_KEY=
```

두 파일의 차이는 다음과 같습니다.

파일	역할	GitHub 공개
`.env`	실제 키와 비밀번호 보관	금지
`.env.example`	필요한 환경변수 이름 안내	가능

쉽게 말하면 `.env`는 실제 열쇠를 보관하는 금고이고, `.env.example`은 금고에 어떤 열쇠가 필요한지 적어 놓은 목록입니다.

② 클라이언트 키와 시크릿 키의 전달 차이

- 클라이언트 키: 서버가 브라우저에 전달해 결제위젯을 실행합니다.
- 시크릿 키: 서버 내부에서만 읽고 결제 승인 API에 사용합니다.

따라서 `/api/config` 같은 설정 API에서 클라이언트 키는 전달할 수 있지만, 시크릿 키는 절대 포함하면 안 됩니다.

4. MCP를 이용해 AI 코딩 에이전트의 지식을 최신화하기

이번 강의에서 특히 중요한 부분입니다.

① MCP는 무엇인가요?

MCP(Model Context Protocol)는 AI 모델이 외부 도구나 자료에 연결해 필요한 정보를 가져오도록 도와주는 연결 방식입니다.

중학생도 이해할 수 있도록 비유하면 다음과 같습니다.

- AI 코딩 에이전트: 개발자
- MCP: 외부 전문가와 연결하는 통로
- 토스페이먼츠 공식 문서: 전문가가 참고하는 최신 업무 매뉴얼

AI가 과거에 학습한 내용만으로 결제 코드를 작성하면 문제가 생길 수 있습니다.

토스페이먼츠 결제위젯의 버전, 함수 이름, 인증 방식 등이 변경될 수 있기 때문입니다.

따라서 MCP를 연결해 AI가 공식 문서를 직접 확인하도록 만드는 것입니다.

② 토스페이먼츠 MCP 연결

강의에서는 Claude Code에 다음 명령을 실행합니다.

```
claude mcp add tosspayments-integration-guide -- npx -y @tosspayments/integration-guid
e-mcp@latest
```

연결한 뒤 Claude Code를 다시 실행하고 다음 명령으로 확인합니다.

```
/mcp
```

여기서 `tosspayments-integration-guide`가 연결되어 있는지 확인합니다.

MCP를 사용할 수 없는 코딩 도구라면 공식 문서의 `llms.txt`를 참고 자료로 제공하는 방법도 있습니다.

③ MCP가 해결하는 것과 해결하지 못하는 것

이 구분이 매우 중요합니다.

구분	MCP가 알려 주는 것	우리가 결정해야 하는 것
결제위젯	실행 방법, SDK, 함수	어떤 화면에 배치할지
결제 승인	API 주소, 인증 방식	승인 전 어떤 조건을 검사할지
결제 오류	오류 코드와 대응 방법	사용자에게 어떤 안내를 할지
크레딧	PG 연동 규칙	상품별 크레딧 수량
주문	결제 관련 기술 규칙	주문 DB 구조와 상태 관리

즉, MCP는 결제 기술을 알려 주지만 우리 사업의 정책과 서비스 구조까지 대신 결정해 주지는 않습니다.

이것은 AI 서비스를 기획하는 사람에게 매우 중요한 역량입니다.

기술적인 세부 사항은 AI에게 맡기더라도, 사업의 규칙과 운영 원칙은 서비스 기획자가 결정해야 하기 때문입니다.

5. 결제 시스템의 전체 구조 설계

오늘 강의에서는 결제 기능을 화면, 서버 API, 데이터베이스로 나누어 설계했습니다.

결제 시스템 구성도

사용자 화면 (static/index.html)

크레딧 안내 팝업 · 상품 선택 · 결제위젯

우리 서버 (API)

- `GET /api/products` : 상품 목록
- `POST /api/orders` : 주문 생성
- `POST /api/payments/confirm` : 결제 승인 및 크레딧 지급

Supabase 데이터베이스

`orders` 테이블 · `credit_ledger` 테이블

토스페이먼츠

결제 인증 · 결제 승인 · 결제 결과 반환

주요 구성 요소별 역할

구성 요소	역할
`GET /api/products`	서버가 관리하는 상품 목록을 화면에 전달
`orders`	결제 주문 정보와 처리 상태 저장
`POST /api/orders`	상품 ID를 받아 서버가 금액과 크레딧을 결정하고 주문 생성
크레딧 안내 팝업	잔액, 상품, 차감 규칙, 환불 안내
결제 화면	클라이언트 키로 결제위젯 실행
성공 페이지	결제 결과를 서버로 전달
`POST /api/payments/confirm`	주문 검증, PG 승인, 크레딧 지급
실패 페이지	결제 오류 안내

여기서 기억할 핵심은 상품의 가격과 지급할 크레딧 수량은 서버가 관리한다는 것입니다.

예를 들어 화면에서 5,000원 상품을 선택했다고 해서 화면이 보내는 금액 5,000원을 그대로 믿는 것이 아닙니다.

화면은 상품 ID만 보내고, 서버가 자기 상품표에서 실제 가격과 크레딧을 찾아야 합니다.

6. 크레딧 안내 팝업이 중요한 이유

결제 시스템은 안전하기만 하면 되는 것이 아닙니다. 사용자가 언제, 왜, 얼마를 결제해야 하는지 이해할 수 있어야 합니다.

오늘 강의에서는 결제위젯을 띄우기 전에 크레딧 안내 팝업을 만들도록 설계했습니다.

Eddie AI v4: Avid support, C2PA signing

redsharknews.com

팝업을 언제 보여 줄까요?

상황	화면 동작
사용자가 크레딧 잔액을 누름	팝업 열기
크레딧 구매 버튼을 누름	팝업 열기
AI 메시지 전송 시 크레딧 부족(HTTP 402)	안내 후 팝업 열기
잔액이 10개 이하	입력창 위에 충전 안내 표시

특히 중요한 사용자 경험은 크레딧이 부족해서 메시지 전송이 거절되었을 때입니다.

사용자가 작성한 메시지를 지우지 않고 그대로 입력창에 남겨 두어야 합니다.

충전한 다음 다시 전송할 수 있기 때문입니다.

팝업에 반드시 포함할 내용

1. 현재 남은 크레딧
2. 메시지당 차감 규칙
3. 상품별 가격과 크레딧 수량
4. 크레딧 1개당 가격
5. 결제 및 환불 안내
6. 게스트 사용자의 데이터 보존 주의사항

이 중에서 특히 주의할 부분은 상품 목록과 잔액을 화면에서 임의로 계산하지 않고 서버가 제공한 값을 그대로 표시하는 것입니다.

7. 결제 완료 후 세 곳의 기록을 비교해야 합니다

이번 강의에서 운영 관점으로 가장 중요하게 배울 내용입니다.

결제가 성공하면 같은 사실이 세 곳에 기록됩니다.

결제 기록의 3중 확인

① 토스페이먼츠

결제 승인 결과, 결제 금액, 결제 키

② orders 테이블

주문번호, 상품, 금액, 완료 상태, 승인 시각

③ credit_ledger 테이블

주문번호를 참조로 한 크레딧 증가 기록

세 곳의 기록이 모두 일치해야 합니다

예를 들어 5,000원 결제로 크레딧 550개를 구매했다고 가정하겠습니다.

확인 장소	정상 기록
PG사	5,000원 승인 성공
orders	해당 주문 완료, 금액 5,000원
credit_ledger	`order:주문번호`, +550

그런데 다음과 같은 문제가 발생할 수 있습니다.

- PG사 결제는 성공했는데 `orders`는 대기 상태
- `orders`는 완료인데 `credit_ledger`에 크레딧 지급 기록이 없음
- 크레딧이 두 번 지급됨

이런 문제를 찾아서 서로 다른 시스템의 기록을 맞추는 작업을 **대사(Reconciliation)**라고 합니다.

쉽게 말해 은행 통장과 회사 장부, 실제 매출 내역을 서로 비교하는 것입니다.

오늘 사용하는 문서용 테스트 키는 개발자센터에 결제 내역이 남지 않기 때문에 승인 응답과 `approved_at`을 확인합니다.

실제 상점 키를 발급받으면 PG사의 결제 조회 API 등을 활용해 확인할 수 있습니다.

8. staging 환경으로 배포하기

로컬 컴퓨터에서 결제가 정상 작동하면 다음에는 staging 환경에서도 확인해야 합니다.

staging은 실제 사용자에게 공개하기 전에 배포 환경에서 기능을 시험하는 공간입니다.

로컬과 staging의 키 관리 차이

환경	시크릿 키 보관	클라이언트 키 보관
내 컴퓨터	`.env`	환경변수 또는 `.env`
staging	Google Secret Manager	일반 환경변수
production	Google Secret Manager	일반 환경변수

강의에서는 다음과 같이 staging 환경을 구성합니다.

- Secret Manager에 `toss-secret-staging` 생성
- `friend-agent-runtime` 서비스 계정에 해당 시크릿 읽기 권한 부여
- `friend-agent-staging` 서비스에 시크릿 연결
- 클라이언트 키는 일반 환경변수로 설정

여기서 중요한 것은 서비스 계정에 모든 시크릿을 읽을 수 있는 권한을 주는 것이 아니라 필요한 시크릿 하나만 읽을 수 있도록 최소 권한을 부여하는 것입니다.

또한 staging의 결제 성공 주소와 실패 주소는 로컬 주소가 아니라 staging 서비스 주소를 사용해야 합니다.

9. 오늘 강의에서 배운 핵심 개념 정리

개념	쉬운 설명	기억할 점
테스트 키	실제 돈 없이 결제를 시험하는 키	문서용 키와 내 상점 키는 다름
`.env`	비밀 설정값을 보관하는 파일	GitHub에 올리지 않음
Secret Manager	클라우드에서 비밀값을 안전하게 관리	배포 환경에서 사용
MCP	AI가 외부 도구와 공식 문서에 연결하는 통로	최신 공식 문서를 참고
API	화면과 서버가 데이터를 주고받는 약속	기능별로 역할을 구분
`orders`	결제 주문 기록	결제 상태 관리
`credit_ledger`	크레딧 거래 원장	충전과 사용 이력 관리
staging	실제 서비스 전 시험 환경	배포 환경에서도 검증
reconciliation	서로 다른 결제 기록을 비교	세 곳의 기록이 일치해야 함

10. MSM과 ASP에 적용한다면?

오늘 배운 구조는 사용자가 준비 중인 두 사업 모두에 직접 활용할 수 있습니다.

MSM(My-Story-Maker)

MSM은 AI가 사용자의 글을 분석하고 웹소설이나 웹툰용 콘텐츠로 변환하는 서비스입니다.

기능	결제·크레딧 설계
글 분석	기본 크레딧 차감
문장 다듬기	소량 차감
웹소설 변환	상대적으로 많은 크레딧 차감

기능	결제·크레딧 설계
웹툰 장면 생성	작업량에 따라 차감
크레딧 충전	PG 결제 후 원장에 지급

MSM에서는 사용자가 자신의 글을 입력하는 과정에서 작업이 길어질 수 있습니다. 따라서 단순히 메시지당 1개를 차감하는 것보다 기능별 사용 원가를 측정하고 차등 차감하는 구조가 적합할 수 있습니다.

ASP(AI Story Partner)

ASP는 AI를 활용해 사업 아이디어 분석, 시장조사, 경쟁사 분석, 사업계획서 작성을 지원하는 서비스입니다.

예를 들어 다음과 같이 구성할 수 있습니다.

서비스	크레딧 차감 예시
사업 아이디어 진단	10
경쟁사 분석	20
시장 분석	30
사업계획서 작성	100
종합 컨설팅 보고서	150

이 경우에도 결제 승인과 크레딧 지급은 동일한 구조를 사용합니다.

장기적으로는 결제 시스템을 공통 모듈로 만들어 MSM과 ASP가 함께 사용할 수 있도록 설계하는 것도 좋은 방법입니다.

11. 이번 강의에서 반드시 기억할 것

1. 테스트 키로 먼저 검증합니다. 실제 돈을 쓰기 전에 결제 전체 흐름을 테스트해야 합니다.
2. 비밀 키는 코드와 분리합니다. `.env`와 Secret Manager를 사용합니다.
3. MCP로 공식 문서를 확인합니다. AI 코딩 에이전트가 오래된 결제 API를 사용하지 않도록 합니다.
4. 사업 규칙은 직접 결정합니다. 상품 가격, 크레딧 지급량, 환불 정책은 MCP가 결정해 주지 않습니다.
5. 결제 기록 세 곳을 비교합니다. PG사, 주문 테이블, 크레딧 원장이 서로 일치해야 합니다.
6. 로컬에서 끝내지 않습니다. staging 환경에서도 결제 흐름을 검증해야 합니다.

한 문장 요약

오늘 강의는 테스트 키와 MCP를 활용해 AI 코딩 에이전트가 공식 결제 규칙을 참고하도록 하고, 주문·결제·크레딧 지급을 안전하게 연결하는 실제 서비스의 결제 시스템 구축 방법을 배운 시간입니다.

다음 단계에서 특히 중요한 학습 포인트

이번 챕터를 이해하셨다면 다음에는 단순히 결제 화면이 뜨는지만 확인하지 마시고, 다음 세 가지를 직접 점검해 보시는 것이 좋습니다.

- 결제가 승인되지 않았는데 크레딧이 지급될 가능성은 없는가?
- 같은 주문을 두 번 처리해도 크레딧이 중복 지급되지 않는가?
- 결제는 성공했는데 크레딧 지급이 실패하면 어떻게 복구할 것인가?

이 세 가지에 답할 수 있다면 결제 기능을 단순히 구현하는 수준을 넘어 실제 서비스 운영 관점에서 이해하신 것입니다.

Chapter07

챕터 07. 같은 요청이 두 번 오는 이유 – 결제 멱등성(Idempotency)

1. 오늘 강의의 목적

오늘 강의의 핵심은 같은 결제 요청이 여러 번 들어오더라도 실제 결제와 크레딧 지급은 한 번만 이루어지도록 만드는 것입니다.

지난 챕터에서는 토스페이먼츠를 이용해 결제 시스템을 구현했습니다.

그런데 실제 서비스를 운영하다 보면 다음과 같은 상황이 발생합니다.

- 사용자가 결제 성공 페이지를 새로고침합니다.
- 네트워크가 느려 브라우저가 같은 요청을 다시 보냅니다.
- 사용자가 결제 버튼을 두 번 누릅니다.
- 서버가 PG사에 승인 요청을 보냈지만 응답을 받기 전에 연결이 끊깁니다.

이런 상황에서 아무런 보호 장치가 없다면 같은 결제 요청이 두 번 처리되어 크레딧이 중복 지급될 수 있습니다.

오늘은 이 문제를 해결하는 **멱등성(Idempotency)**과 이를 구현하는 세 가지 안전장치를 배웁니다.

2. 멱등성(Idempotency)이란 무엇인가요?

① 멱등성의 의미

멱등성이란 같은 요청을 여러 번 실행해도 결과가 한 번 실행한 것과 동일하게 유지되는 성질입니다.

쉽게 비유하면 엘리베이터 버튼을 생각하시면 됩니다.

1층 버튼을 한 번 누르든 세 번 누르든 엘리베이터가 1층으로 가는 결과는 같습니다.

반면 크레딧 지급 버튼을 세 번 눌렀을 때 크레딧이 세 배가 된다면 문제가 됩니다.

요청	멱등성이 없는 경우	멱등성이 있는 경우
최초 결제 승인 요청	+550	+550
같은 요청 2회차	+550	변화 없음
같은 요청 3회차	+550	변화 없음
최종 크레딧 증가	+1,650	+550

같은 결제 요청은 여러 번 들어와도 실제 크레딧 지급은 한 번만 이루어져야 합니다.

② 왜 같은 요청이 두 번 들어올까요?

가장 이해하기 쉬운 사례는 네트워크 문제입니다.

서버와 PG사 사이에서 발생하는 문제

우리 서버

결제 승인 요청 전송

토스페이먼츠

실제 결제 승인 완료

응답 전달 중 네트워크 오류!

우리 서버

승인 여부를 모름 → 같은 요청 재전송

이때 토스페이먼츠는 이미 결제를 승인했지만 우리 서버는 결과를 모를 수 있습니다.

따라서 같은 승인 요청을 다시 보내야 합니다.

이때 필요한 것이 바로 멱등키입니다.

3. 중복 결제를 막는 세 겹의 안전장치

오늘 강의의 가장 중요한 내용입니다.

결제 중복을 막기 위해 세 가지 장치를 함께 사용합니다.

결제 중복 방지 3중 안전장치

1차 방어 · 우리 서버

주문 상태 확인

이미 완료된 주문이면 승인 API를 다시 호출하지 않습니다.

2차 방어 · 데이터베이스

원장 참조값의 유일성

같은 주문번호로 크레딧을 두 번 지급하려 하면 DB가 중복 기록을 거절합니다.

3차 방어 · PG사

Idempotency-Key

같은 키로 승인 API를 다시 호출하면 토스페이먼츠가 첫 응답을 돌려주고 중복 처리를 방지합니다.

각각의 역할을 좀 더 자세히 살펴보겠습니다.

① 첫 번째: 주문 상태 확인

우리 서버의 `orders` 테이블에는 주문 상태가 기록됩니다.

주문번호	금액	상태
ORDER-001	5,000원	대기
ORDER-002	10,000원	완료

사용자가 ORDER-002의 성공 페이지를 다시 열면 서버는 주문 상태를 확인합니다.

이미 완료된 주문이므로 결제 승인 API를 다시 호출하지 않습니다.

즉, 이미 처리한 주문을 다시 처리하지 않는 장치입니다.

다만 두 요청이 거의 동시에 들어오면 문제가 생길 수 있습니다.

두 요청이 모두 주문 상태를 확인했을 때 아직 대기 상태라면 둘 다 승인 처리를 시작할 수 있기 때문입니다.

따라서 주문 상태 확인만으로는 충분하지 않습니다.

② 두 번째: 원장 참조값의 유일성

지난 챕터에서 배운 `credit_ledger`를 다시 활용합니다.

결제 충전이 발생하면 다음과 같은 기록을 남깁니다.

```
user_id: user-001
amount: +550
reason: purchase
reference: order:ORDER-001
```

여기서 중요한 것은 `reference`입니다.

`order:ORDER-001`이라는 참조값은 한 번만 사용할 수 있도록 DB에 Unique 제약조건을 설정합니다.

따라서 같은 주문으로 크레딧을 두 번 지급하려고 하면 데이터베이스가 두 번째 기록을 거절합니다.

원장은 크레딧이 실제로 두 번 지급되는 것을 막는 최종 방어선 중 하나입니다.

단, Unique 제약조건만으로 결제 승인 중복까지 막을 수 있는 것은 아닙니다. 결제 처리와 크레딧 지급을 원자적으로 관리해야 합니다.

③ 세 번째: 토스페이먼츠 Idempotency-Key

토스페이먼츠에 승인 요청을 보낼 때 `Idempotency-Key`라는 헤더를 함께 보냅니다.

이 값은 같은 승인 요청을 구분하기 위한 고유한 식별자입니다.

예를 들어 다음과 같습니다.

```
Idempotency-Key: ORDER-001
```

같은 주문에 대해 다시 승인 요청을 보내더라도 동일한 키를 사용합니다.

그러면 토스페이먼츠는 같은 키, 같은 API 키, 같은 주소로 들어온 요청에 대해 첫 응답을 재사용합니다.

강의 자료 기준으로 멱등키는 300자 이하이며, 최초 사용일부터 15일간 유효합니다.

특히 기억해야 할 내용은 다음과 같습니다.

- 같은 주문은 항상 같은 멱등키를 사용합니다.
- 요청할 때마다 새로운 무작위 값을 만들면 안 됩니다.
- 처리 중이라는 409 응답을 받으면 같은 키로 다시 요청합니다.
- 오류가 발생했다고 멱등키를 바꿔서 재시도하면 안 됩니다.

4. 멱등키는 어떻게 만들어야 할까요?

① 주문번호를 멱등키로 사용할 수 있습니다

강의에서는 주문번호를 멱등키로 사용하는 방식을 설명합니다.

예를 들어 다음과 같습니다.

```
주문번호: ORDER-001
멱등키: ORDER-001
```

같은 주문의 승인 요청이라면 항상 같은 키를 사용합니다.

주문번호 자체가 추측하기 어려운 무작위 값으로 생성되어 있다면 이 방법을 사용할 수 있습니다.

또는 주문 생성 시 별도의 무작위 멱등키를 만들어 DB에 저장해도 됩니다.

② 잘못된 방법과 올바른 방법

구분	잘못된 방법	올바른 방법
키 생성 시점	승인 요청 때마다 생성	주문 생성 시 결정
동일 주문 재요청	새로운 키 사용	기존 키 재사용
네트워크 오류	다른 키로 재시도	같은 키로 재시도
주문이 다른 경우	같은 키 재사용	주문별 고유 키 사용

이것은 결제 시스템뿐 아니라 AI 에이전트가 외부 API를 호출할 때도 중요한 개념입니다.

예를 들어 AI 에이전트가 이메일 발송 요청을 했는데 응답이 끊겼다고 생각해 보겠습니다.

같은 이메일을 다시 보내면 사용자에게 이메일이 두 통 전달될 수 있습니다.

이런 문제를 방지하기 위해 외부 서비스 요청에도 멱등성을 적용할 수 있습니다.

5. 직접 테스트해야 하는 두 가지 상황

오늘 강의에서는 실제 브라우저에서 두 가지 상황을 확인합니다.

① 같은 승인 요청을 두 번 보내기

테스트 방법은 다음과 같습니다.

1. 테스트 결제를 정상적으로 완료합니다.
2. 충전 완료 페이지를 새로고침합니다.

3. 같은 주문의 승인 요청이 다시 발생하는지 확인합니다.
4. 크레딧 잔액과 원장 기록을 비교합니다.

정상 결과는 다음과 같습니다.

확인 항목	기대 결과
승인 요청 로그	여러 번 발생할 수 있음
Idempotency-Key	모든 요청에서 동일
최종 잔액	최초 충전한 만큼만 증가
credit_ledger	+550 기록 한 줄
주문 상태	완료 유지

화면에 충전 완료 메시지가 다시 표시되는 것은 문제가 아닙니다.

이미 완료된 주문에 대해 기존 결과를 다시 보여 주는 것은 정상적인 동작입니다.

중요한 것은 잔액이 추가로 증가하지 않는다는 점입니다.

② 결제 취소

두 번째 테스트는 결제 취소입니다.

1. 결제위젯을 실행합니다.
2. 결제를 진행하다 취소합니다.
3. 실패 페이지로 이동하는지 확인합니다.
4. 주문 상태가 실패로 변경되는지 확인합니다.
5. 크레딧 원장에 추가 기록이 없는지 확인합니다.

결제 취소는 결제 실패와 구분해서 관리할 수도 있습니다. 실제 서비스에서는 주문 상태를 대기·완료·실패·취소 등으로 세분화할지 정책을 정해야 합니다.

6. 자동화 테스트로 네 가지 상황을 검증하기

브라우저에서 직접 확인하는 것만으로는 모든 결제 오류 상황을 재현하기 어렵습니다.

예를 들어 다른 사용자의 주문을 승인하려는 상황이나 결제 금액을 조작하는 상황은 자동화 테스트로 확인하는 것이 좋습니다.

네 가지 테스트 시나리오

테스트 상황	검증할 내용
같은 승인 요청 두 번	크레딧이 한 번만 지급되는가?
금액을 바꾼 요청	PG 승인 API가 호출되지 않는가?
다른 사용자의 주문	본인 소유가 아닌 주문을 승인할 수 없는가?
승인 실패	주문이 완료되지 않고 크레딧도 지급되지 않는가?

특히 금액 조작과 다른 사용자의 주문 테스트에서는 가짜 승인 API가 한 번도 호출되지 않았는지 확인해야 합니다.

왜 가짜 승인 API를 사용할까요?

지난 강의에서 가짜 LLM으로 테스트했던 것과 같은 원리입니다.

실제 토스페이먼츠를 호출하지 않고, 테스트 코드 안에서 가짜 승인 응답을 반환하도록 만듭니다.

이렇게 하면 다음과 같은 장점이 있습니다.

- 실제 결제 요청을 반복하지 않아도 됩니다.
- 테스트 비용이 발생하지 않습니다.
- 매번 같은 결과를 얻을 수 있습니다.
- 결제 실패와 오류 상황을 자유롭게 재현할 수 있습니다.

테스트 실행 명령은 다음과 같습니다.

```
pytest -q
```

테스트 결과가 모두 통과하면 네 가지 상황에서 정해 둔 규칙이 지켜지는지 자동으로 확인할 수 있습니다.

7. 이번 강의에서 특히 주의해야 할 기술적 내용

여기서는 강의 내용을 실제 서비스 운영 관점에서 한 단계 더 이해할 필요가 있습니다.

동시에 요청이 들어오는 문제

예를 들어 같은 주문에 대해 두 요청이 거의 동시에 들어왔다고 가정하겠습니다.

시점	요청 A	요청 B
1	주문 상태 확인	
2	대기 상태 확인	

시점	요청 A	요청 B
3		주문 상태 확인
4		대기 상태 확인
5	승인 요청	승인 요청
6	크레딧 지급	크레딧 지급 시도

두 요청이 모두 대기 상태를 읽었다면 주문 상태 확인만으로 중복 처리를 막을 수 없습니다.

이때는 다음과 같은 기술이 추가로 필요합니다.

- DB 트랜잭션
- 원자적 상태 변경(예: 대기 상태일 때만 처리 중으로 변경)
- Unique 제약조건
- 토스페이먼츠 멱등키

특히 승인 처리를 시작할 때 주문 상태를 원자적으로 `pending`에서 `processing`으로 바꾸는 방식이 유용합니다.

```
대기(pending)
      ↓
처리 중(processing)
      ↓
완료(completed)
```

다른 요청이 동시에 들어오더라도 이미 `processing`인 주문은 다시 처리하지 않도록 만드는 것입니다.

또한 PG사 승인은 외부 시스템에서 일어나므로 DB 트랜잭션 하나로 PG사와 우리 DB의 모든 작업을 묶을 수는 없습니다. 따라서 승인 결과를 복구할 수 있는 재시도 및 대사 절차도 함께 설계해야 합니다.

8. MSM과 ASP에 적용한다면?

오늘 배운 멱등성은 사용자가 준비하는 서비스에도 매우 중요합니다.

MSM(My-Story-Maker)

MSM에서 다음과 같은 상황을 생각해 보겠습니다.

사용자가 자신의 글을 웹소설로 변환하는 버튼을 눌렀는데 응답이 늦게 옵니다.

사용자는 버튼을 다시 누릅니다.

그런데 두 요청이 모두 실행되면 다음 문제가 생길 수 있습니다.

- 동일한 웹소설이 두 번 생성됩니다.
- AI API 비용이 두 번 발생합니다.
- 크레딧이 두 번 차감됩니다.
- 동일한 결과물이 두 개 저장됩니다.

따라서 MSM에서도 작업별 고유 ID를 만들고, 같은 작업 요청은 한 번만 실행되도록 설계하는 것이 좋습니다.

예를 들면 다음과 같습니다.

```
작업 ID: MSM-WEBNOVEL-001
사용자: user-001
작업 유형: 웹소설 변환
차감 크레딧: 5
작업 상태: processing
```

같은 작업 ID로 요청이 다시 들어오면 새 작업을 만들지 않고 기존 작업의 진행 상태나 결과를 돌려줍니다.

ASP(AI Story Partner)

ASP에서도 마찬가지입니다.

예를 들어 고객이 100크레딧을 사용해 종합 사업계획서 생성을 요청했다고 가정하겠습니다.

네트워크 문제로 요청이 반복되면 다음 문제가 발생할 수 있습니다.

- AI 보고서가 중복 생성됩니다.
- LLM API 비용이 중복 발생합니다.
- 크레딧이 중복 차감됩니다.

따라서 작업 ID를 기준으로 요청과 비용 차감을 관리해야 합니다.

결제에서 배운 멱등성은 AI 에이전트가 외부 API를 호출하거나 시간이 오래 걸리는 작업을 처리할 때도 활용할 수 있는 핵심 설계 원칙입니다.

9. 오늘 강의 핵심 내용 정리

핵심 개념	반드시 기억할 내용
멱등성	같은 요청을 여러 번 보내도 결과는 한 번 처리한 것과 같음
주문 상태 확인	완료된 주문은 다시 승인하지 않음

핵심 개념	반드시 기억할 내용
원장 참조 유일성	동일 주문의 크레딧 중복 지급 방지
Idempotency-Key	PG사 승인 API의 중복 처리 방지
동일 키 재사용	같은 주문의 재시도는 항상 같은 키
409 응답	처리 중이면 잠시 기다린 뒤 같은 키로 재시도
자동화 테스트	중복 요청, 금액 조작, 타인 주문, 승인 실패 검증
DB 원자성	동시에 요청이 들어와도 한 요청만 처리하도록 보장

10. 주관식 퀴즈 모범 답안

핵심 정답

질문: 같은 승인 요청이 거의 동시에 두 번 들어와서 두 요청 모두 주문 상태를 대기로 읽었습니다. 이 경우 크레딧이 두 번 들어가지 않으려면 어느 장치가 막아 주나요?

모범 답안:

두 요청이 거의 동시에 들어오면 주문 상태 확인만으로는 중복 처리를 막을 수 없습니다. 두 요청 모두 주문 상태를 대기로 읽었기 때문입니다.

이때 데이터베이스의 `credit_ledger`에 설정한 참조값의 Unique 제약조건이 같은 주문번호로 크레딧 지급 기록을 두 번 만드는 것을 막아 줍니다.

또한 토스페이먼츠의 `Idempotency-Key`는 같은 승인 요청이 반복되더라도 결제가 중복 처리되지 않도록 보호합니다.

만약 원장 참조값의 유일성 장치가 없다면 두 요청이 각각 크레딧을 지급해 550개가 두 번 들어갈 수 있습니다.

실제 운영에서는 여기에 DB의 원자적 상태 변경과 트랜잭션까지 적용해 중복 처리를 방지해야 합니다.

11. 한 문장으로 정리

멱등성은 네트워크 오류나 사용자 중복 클릭으로 같은 요청이 여러 번 들어오더라도 결제와 크레딧 지급이 한 번만 이루어지도록 보장하는 안전장치입니다.

이번 챕터에서 특히 중요한 것은 세 겹의 방어 구조입니다.

- 우리 서버는 주문 상태를 확인합니다.
- 데이터베이스는 원장 중복 기록을 차단합니다.
- PG사는 멱등키로 중복 승인 처리를 방지합니다.

이 원칙을 이해하면 MSM과 ASP를 비롯한 AI 서비스에서 중복 작업, 중복 비용, 중복 결제를 방지하는 설계 역량까지 갖추게 됩니다.

Chapter08

/강의정리 챕터 08. AI 서비스의 원가 관리와 수익성 설계

1. 오늘 강의의 목적

오늘 강의의 핵심은 AI 서비스를 만들 수 있는가에서 한 걸음 더 나아가, 그 서비스를 지속적으로 운영하면서 돈을 벌 수 있는가를 판단하는 것입니다.

AI 서비스는 일반적인 웹서비스와 달리 사용자가 서비스를 이용할 때마다 LLM 호출 비용이 발생합니다. 사용자가 많아지면 매출도 늘지만, 비용도 함께 늘어날 수 있습니다.

따라서 오늘은 다음 네 가지를 학습했습니다.

1. 사용자 한 명이 서비스를 이용할 때 실제로 얼마가 드는가?
2. 얼마를 무료로 제공하고, 얼마를 유료로 받아야 하는가?
3. 크레딧과 결제를 어떻게 안전하게 관리하는가?
4. 일부 사용자가 과도하게 이용해도 손실을 통제할 수 있는가?

2. 핵심 개념 쉽게 이해하기

① AI 서비스의 원가는 사용량에 비례합니다

일반적인 웹사이트는 사용자가 페이지를 한 번 더 열었다고 해서 비용이 크게 늘지는 않습니다.

하지만 AI 서비스는 다릅니다.

사용자가 질문 하나를 보내면 내부적으로 여러 번 AI를 호출할 수 있습니다.

예를 들어 사용자가 AI에게 사업계획서 작성을 요청했다고 가정하겠습니다.

사용자의 메시지 한 건

사업계획서를 작성해 주세요

1차 호출

요청 분석

2차 호출

시장 조사

3차 호출

내용 작성

4차 호출

결과 검토 및 수정

최종 답변 제공

4번의 호출 비용을 합산한 것이 메시지 한 건의 원가

즉, 사용자가 보낸 메시지 수만으로는 원가를 정확하게 계산할 수 없습니다.

- LLM을 몇 번 호출했는가?
- 각 호출에서 입력 토큰과 출력 토큰을 얼마나 사용했는가?
- 어떤 모델을 사용했는가?
- 대화가 길어지면서 이전 대화 내용까지 얼마나 다시 전달했는가?

이런 정보를 종합해야 실제 원가를 알 수 있습니다.

Langfuse의 역할

Langfuse는 AI 서비스 내부에서 어떤 호출이 발생했고, 각각의 토큰과 비용이 얼마나 들었는지 기록하고 분석하는 도구입니다.

오늘은 Langfuse를 통해 평균 원가뿐 아니라 비용이 많이 발생하는 사용자와 긴 대화의 원가 차이도 확인했습니다.

특히 중요한 사실은 다음과 같습니다.

> 평균적으로 저렴한 서비스라도 일부 사용자가 과도하게 이용하면 전체 수익성이 크게 나빠질 수 있습니다.

② 가격은 원가만으로 결정하지 않습니다

AI 서비스의 가격을 정할 때는 LLM 비용만 계산해서는 안 됩니다.

가격 결정 요소	의미
AI 원가	LLM 호출에 들어가는 비용
결제 수수료	PG사 등에 지급하는 비용
고정비	서버, DB, 도메인 등 운영비
무료 한도	무료 사용자에게 제공하는 사용량

가격 결정 요소	의미
전환율	무료 사용자 중 유료로 전환하는 비율

예를 들어 AI 서비스의 월간 손익을 생각해 보겠습니다.

- 무료 사용자가 많아지면 AI 원가가 증가합니다.
- 무료 한도가 너무 높으면 무료 사용자 원가가 커집니다.
- 무료 한도가 너무 낮으면 서비스 경험이 나빠질 수 있습니다.
- 유료 전환율이 높아지면 무료 사용자의 비용을 감당할 여력이 생깁니다.

따라서 처음부터 무료 한도를 크게 주기보다는 작게 시작하고, 실제 사용량과 유료 전환율을 측정하면서 조정하는 것이 합리적입니다.

또한 고성능 모델로 바꾸면 응답 품질이 좋아질 수 있지만 원가도 증가합니다. 반대로 저렴한 모델로 바꾸거나 호출 횟수를 줄이면 비용은 낮아질 수 있지만 품질이 떨어질 수 있습니다.

그래서 비용을 줄일 때도 반드시 평가를 통해 품질을 확인해야 합니다.

③ 크레딧은 원장으로 관리합니다

크레딧은 AI 서비스를 이용할 수 있는 가상의 포인트입니다.

여기서 중요한 것은 잔액을 직접 덮어쓰지 않는다는 점입니다.

예를 들어 사용자가 1,000크레딧을 충전하고, 100크레딧을 사용했다면 다음처럼 기록합니다.

거래 내용	증감	잔액
최초 충전	+1,000	1,000
AI 서비스 사용	-100	900
추가 충전	+500	1,400
AI 서비스 사용	-200	1,200

이렇게 거래 내역을 하나씩 쌓아 관리하는 방식을 **원장(Ledger)**이라고 합니다.

왜 잔액만 저장하지 않을까요?

잔액만 저장하면 언제, 어떤 이유로 크레딧이 증가하거나 감소했는지 확인하기 어렵습니다. 원장을 사용하면 거래 이력을 추적하고 오류가 발생했을 때 원인을 찾을 수 있습니다.

그리고 각 거래의 참조값을 유일하게 만들어 같은 작업이 두 번 기록되지 않도록 합니다.

④ 결제는 서버에서 승인해야 완료됩니다

결제 과정에서 가장 중요한 것은 브라우저 화면의 성공 메시지가 아니라 서버에서 실제 승인 결과를 확인하는 것입니다.

안전한 결제 흐름

1. 주문 생성

   서버가 상품 가격과 크레딧을 확인하고 주문을 먼저 저장합니다.

2. 결제 인증

   사용자가 PG 결제 화면에서 카드 인증 등을 진행합니다.

3. 서버 금액 검증

   successUrl로 전달된 금액과 서버에 저장된 주문 금액을 비교합니다.

4. PG 승인 요청

   서버가 시크릿 키를 사용해 PG사에 승인 요청을 보냅니다.

5. 크레딧 지급

   승인이 성공한 경우에만 주문을 완료하고 원장에 크레딧을 기록합니다.

여기서 추가로 배운 것이 **멱등성(Idempotency)**입니다.

같은 주문의 승인 요청이 네트워크 문제나 새로고침 때문에 두 번 들어와도 실제 결제와 크레딧 지급은 한 번만 이루어져야 합니다.

이를 위해 다음 세 가지 방어 장치를 사용합니다.

1. 주문 상태 확인: 이미 완료된 주문이면 다시 승인하지 않습니다.
2. 원장 참조값 Unique: 같은 주문으로 크레딧이 두 번 지급되지 않게 합니다.
3. PG 멱등키: 같은 주문의 승인 요청을 재전송해도 PG가 중복 처리하지 않도록 합니다.

특히 같은 주문에는 항상 같은 멱등키를 사용해야 합니다.

3. 오늘 배운 내용이 실제 서비스에서 중요한 이유

오늘의 강의는 AI 서비스의 사업성을 결정하는 핵심 내용입니다.

AI 서비스는 단순히 좋은 결과를 제공한다고 성공하는 것이 아닙니다. 사용자가 서비스를 많이 이용할수록 원가가 증가하기 때문에, 사용량을 관리하고 수익을 확보할 수 있어야 합니다.

일반적인 서비스 운영과 AI 서비스 운영의 차이를 비교하면 다음과 같습니다.

구분	일반적인 웹서비스	AI 서비스
주요 비용	서버·DB·트래픽	서버·DB·LLM 호출
사용량 증가	비용 증가가 상대적으로 완만할 수 있음	AI 호출량에 따라 원가 증가
원가 관리	서버 자원 중심	토큰·호출 횟수·모델 중심
사용자 관리	접속·트래픽 관리	사용량·크레딧·비용 관리
수익성 관리	구독료·광고·판매	사용량에 따른 AI 원가까지 고려

따라서 AI 서비스 기획자는 기능과 화면뿐 아니라 사용자 한 명이 서비스를 이용할 때 얼마의 비용이 발생하는지까지 설계해야 합니다.

4. 나의 프로젝트에 적용하기

① MSM(My-Story-Maker)

MSM은 사용자의 일기나 SNS 글을 분석하고, 웹소설 형태로 재구성하는 서비스입니다.

이 과정에서 여러 차례 AI 호출이 발생할 수 있습니다.

- 원본 글 분석
- 사용자 문체 분석
- 이야기 구조 설계
- 웹소설 생성
- 결과 수정 및 재작성

따라서 단순히 글 한 편을 생성하는 데 몇 크레딧을 받을 것인지가 아니라, 웹소설 한 편을 완성하는 데 평균적으로 얼마의 AI 원가가 발생하는지를 계산해야 합니다.

예를 들어 짧은 글과 긴 글의 원가가 다르다면 입력 글의 길이에 따라 크레딧을 차등 차감하는 방법도 생각할 수 있습니다.

또한 사용자가 같은 변환 버튼을 여러 번 눌렀을 때 AI 작업이 중복 실행되지 않도록 작업 ID와 멱등성을 적용해야 합니다.

② ASP(AI Story Partner)

ASP는 기업이나 개인의 사업 아이디어를 분석하고 보고서와 사업계획서를 작성하는 컨설팅 서비스입니다.

예를 들어 고객이 사업계획서 작성을 요청하면 다음과 같은 작업이 발생할 수 있습니다.

1. 고객 요구사항 분석
2. 시장 및 경쟁사 조사

3. 사업모델 분석
4. 전략 수립
5. 보고서 작성
6. 결과 검토 및 수정

여기서는 모든 고객에게 동일한 크레딧을 차감하는 것보다 작업의 복잡도와 예상 AI 호출량을 기준으로 상품을 구분하는 것이 합리적입니다.

- 기본 분석: 낮은 크레딧
- 심층 분석: 중간 크레딧
- 종합 사업계획서: 높은 크레딧

또한 고비용 모델은 중요한 분석이나 최종 검토에만 사용하고, 단순 분류나 요약은 저렴한 모델로 처리하는 방식도 고려할 수 있습니다.

5. 주관식 퀴즈 모범답안

Q. 하루 300개를 보내는 사용자 때문에 생기는 문제를 오늘 배운 원가표, 크레딧 한도, 결제 흐름이 어디까지 막고, 아직 막지 못하는 것은 무엇인가요?

모범답안

하루 300개의 메시지를 보내는 사용자가 발생하면 일반 사용자보다 LLM 호출량과 토큰 사용량이 크게 증가하여 AI 서비스의 원가가 급격히 상승할 수 있습니다. 특히 한 메시지에 여러 번의 LLM 호출이 발생한다면 실제 비용은 메시지 개수만으로 예상한 것보다 훨씬 커질 수 있습니다.

오늘 만든 원가표와 Langfuse 분석을 통해 사용자별 평균 원가와 고비용 사용자를 파악할 수 있고, 크레딧 한도와 무료 사용량 제한을 통해 사용자가 소비할 수 있는 비용의 상한을 설정할 수 있습니다. 유료 사용자는 결제와 크레딧 충전 과정을 거치게 하고, 서버에서 크레딧을 차감하도록 하여 무제한으로 AI 서비스를 이용하는 것도 방지할 수 있습니다.

또한 주문 원장과 멱등키를 적용하면 같은 결제나 크레딧 지급이 중복 처리되는 문제도 예방할 수 있습니다.

그러나 이러한 장치만으로 모든 문제를 막을 수는 없습니다. 예를 들어 여러 계정을 만들어 무료 한도를 반복해서 사용하는 행위, 자동화 프로그램을 이용한 대량 요청, 긴 입력과 복잡한 AI 작업으로 인한 예상 밖의 원가 증가 등은 추가적인 관리가 필요합니다.

따라서 앞으로는 사용자별 일일·월간 사용량 제한, 요청 속도 제한(Rate Limit), 동시 실행 작업 수 제한, 비정상 사용 패턴 탐지, 작업별 최대 토큰 제한 등을 추가해야 합니다. 필요하

면 사용량이 일정 수준을 넘었을 때 자동으로 요청을 차단하거나 관리자에게 알림을 보내는 기능도 구축해야 합니다.

결론적으로 원가표는 비용을 파악하고, 크레딧 한도는 비용을 통제하며, 결제 시스템은 정당한 과금과 중복 지급을 방지합니다. 여기에 비정상 사용 탐지와 요청 제한까지 결합해야 안정적인 AI 서비스 운영이 가능합니다.

6. 반드시 기억해야 할 핵심 사항

핵심 개념	반드시 기억할 내용
AI 원가	메시지 한 건의 원가는 그 안에서 발생한 모든 LLM 호출 비용의 합
비용 분석	평균 원가뿐 아니라 고비용 사용자와 긴 대화도 분석
가격 정책	원가, 수수료, 고정비, 무료 한도, 유료 전환율을 함께 고려
모델 선택	모델 비용과 응답 품질 사이의 균형
크레딧	잔액을 덮어쓰지 않고 거래 원장으로 관리
결제 승인	PG 승인은 반드시 서버에서 수행
멱등성	같은 요청이 반복되어도 결과는 한 번 처리한 것과 같아야 함
추가 방어	Rate Limit, 일일 한도, 비정상 사용 탐지 필요

7. 이번 강의와 AI 창업 목표의 연결

오늘 강의는 AI 서비스를 만드는 기술을 넘어 AI 서비스를 사업으로 운영하는 방법을 배운 시간입니다.

특히 앞으로 MSM과 ASP를 실제 고객에게 제공하려면 다음 세 가지를 반드시 갖춰야 합니다.

1. 원가를 알아야 합니다. 고객 한 명이 어떤 기능을 얼마나 이용할 때 비용이 발생하는지 알아야 합니다.
2. 사용량을 통제해야 합니다. 무료 체험과 유료 상품의 경계를 설계하고 과도한 사용을 막아야 합니다.
3. 수익을 안전하게 관리해야 합니다. 결제, 크레딧, 사용 기록이 정확하게 연결되어야 합니다.

앞으로는 AI 기능을 하나 추가할 때마다 그 기능의 원가와 판매 가격, 크레딧 차감 기준을 함께 설계하는 습관이 필요합니다.

8. 오늘 강의 한 문장 요약

AI 서비스의 성공은 좋은 답변을 만드는 것에서 끝나지 않으며, 사용자별 AI 원가를 측정하고 사용량을 통제하며 안전하게 과금할 수 있는 수익 구조를 만드는 데 달려 있습니다.